

Module 1.1: Introduction to Big Data

Contents

Big Data	1
Characteristics of Big Data (The 5 Vs)	1
Why Big Data?	3
Types of Data	4
Applications of Big Data	6
Industry Examples of Big Data	6
Web Analytics	6
Big Data in Marketing	7
Fraud Detection using Big Data	7
Risk Analytics using Big Data	7
Credit Risk Management	8
Big Data and Algorithmic Trading	8
Big Data in Healthcare	8
Advertising and Big Data	9
Big Data Technologies	9

Big Data

Big Data refers to extremely large, complex, and rapidly growing datasets that cannot be effectively stored, processed, managed, or analyzed using traditional database systems and software tools.

Traditional databases are designed to handle structured data of limited size. However, with the growth of the internet, social media, mobile devices, cloud computing, and IoT, enormous amounts of data are generated every second. This massive volume of data requires specialized technologies and techniques for storage, processing, and analysis.

Sources of Big Data

- Social media platforms (Facebook, Instagram, X)
- E-commerce websites (Amazon, Flipkart)
- Search engines (Google)
- Banking transactions
- Healthcare systems
- IoT devices and sensors
- Mobile applications
- Scientific research

Examples of Big Data

- YouTube receives hundreds of hours of video uploads every minute.
- Facebook users generate millions of posts, comments, and likes daily.
- Online shopping platforms record millions of customer transactions.
- Hospitals store huge volumes of patient records and medical images

Characteristics of Big Data (The 5 Vs)

Big Data is commonly described using five important characteristics known as the **5 Vs**.

1. **Volume:** Volume refers to the enormous quantity of data generated and stored. As organizations collect information from various sources, the amount of data grows rapidly, reaching terabytes, petabytes, and even exabytes. **Examples**

- Social media posts
- Online transactions
- Video streaming data
- Sensor-generated data

Importance: Large volumes of data require scalable storage systems and distributed processing techniques. **Example:** A large e-commerce company may store several petabytes of customer transaction data.

2. **Velocity:** Velocity refers to the speed at which data is generated, transmitted, and processed. Modern applications often require real-time or near real-time analysis. **Examples**

- Stock market transactions
- Credit card payments
- GPS tracking systems
- Social media updates

Importance: Organizations must process data quickly to make timely decisions. **Example:** Banks analyze credit card transactions instantly to detect fraud.

3. **Variety:** Variety refers to the different types and formats of data. Unlike traditional databases that mainly handle structured data, Big Data includes multiple forms of information. **Types of Data**

a) **Structured Data:** Data organized in rows and columns. Examples:

- Employee databases
- Student records

b) **Semi-Structured Data:** Data that contains tags or markers but does not follow a rigid structure. Examples:

- XML
- JSON

c) **Unstructured Data:** Data without a predefined format. Examples:

- Images
- Videos
- Audio files
- Emails
- Social media posts

Importance

Organizations must manage and analyze data from multiple sources and formats.

4. **Veracity:** Veracity refers to the quality, reliability, and accuracy of data. Not all collected data is correct or trustworthy. Poor-quality data can lead to incorrect conclusions. **Problems Affecting Veracity**

- Missing values
- Duplicate records
- Inconsistent information
- Fake or misleading data

Importance: Reliable data improves decision-making and analytical accuracy. **Example:** Fake social media accounts may generate inaccurate information that affects business analysis

5. **Value:** Value refers to the useful insights and benefits extracted from data. Collecting large amounts of data is meaningless unless organizations can derive valuable information from it.

6. **Importance:** Big Data helps organizations:

- Improve decision-making
- Increase profits

- Understand customers
- Optimize operations

Example: Netflix analyzes user viewing habits to recommend movies and TV shows.

	Meaning
Volume	Amount of data
Velocity	Speed of data generation and processing
Variety	Different forms of data
Veracity	Accuracy and reliability of data
Value	Useful insights obtained from data



Why Big Data?

Traditional database systems were designed to manage relatively small amounts of structured data. As technology advanced, data began growing at an unprecedented rate. Conventional systems became insufficient for handling this massive volume and complexity.

Big Data technologies were developed to overcome these limitations.

Reasons for the Emergence of Big Data

- **Explosion of Digital Data:** The rapid growth of digital devices and online services continuously generates huge amounts of data from social media, mobile apps, e-commerce platforms, and sensors.
- **Growth of the Internet:** Billions of users interact online daily, creating vast amounts of information through searches, website visits, and online transactions.
- **Rise of social media:** Social networking platforms generate massive volumes of user-created content such as posts, comments, images, and videos.
- **Cloud Computing:** Cloud platforms provide scalable and cost-effective storage and computing resources, making it easier to handle large datasets.
- **Mobile Computing:** Smartphones and mobile applications continuously generate data related to user activities, location, communication, and online services.
- **Internet of Things (IoT):** Connected devices like smart watches, smart homes, and industrial sensors continuously collect and transmit data.
- **Need for Real-Time Analytics:** Organizations require immediate analysis of data for applications such as fraud detection, traffic management, and recommendation systems.
- **Advances in Storage Technology:** Storage has become cheaper and more accessible, enabling organizations to store massive amounts of data.
- **Competitive Business Environment:** Organizations use data-driven strategies to gain competitive advantages, improve decision-making, and enhance customer satisfaction.

Advantages of Big Data

1. **Better Decision Making:** Helps organizations make accurate decisions using historical and real-time data.
2. **Improved Customer Experience:** Enables businesses to understand customer preferences and provide personalized services.
3. **Fraud Detection:** Identifies suspicious activities and helps prevent fraud in real time.

4. **Risk Management:** Assists organizations in identifying and reducing potential risks.
5. **Increased Efficiency:** Optimizes business processes and improves resource utilization.
6. **Innovation:** Supports the development of new products, services, and business strategies.

Limitations and Challenges of Big Data

1. Storage Complexity
2. Data Security Issues
3. Privacy Concerns
4. Data Quality Problems
5. High Processing Requirements
6. Skilled Workforce Requirements

Types of Data

Data can be classified into three main categories based on its structure and organization: **Structured Data, Semi-Structured Data, and Unstructured Data**. Understanding these types is important because Big Data systems are designed to handle all forms of data.

1. **Structured Data:** is data that is organized in a predefined format, usually in rows and columns. It follows a fixed schema, making it easy to store, search, and analyze using traditional systems DBMS.

- Organized in tabular form.
- Follows a predefined schema.
- Easy to store and retrieve.
- Can be managed using SQL databases.
- Highly structured and consistent.

Example Table

Student ID	Name	Marks
101	Amit	85
102	Rahul	90

Advantages

- Easy querying and analysis.
- High data integrity.
- Efficient storage and retrieval.

Limitations

- Difficult to handle complex and changing data formats.
- Not suitable for multimedia content.

Examples

- Student records
- Employee databases
- Banking transactions
- Sales records

2. **Semi-Structured Data:** does not follow a rigid table structure but contains tags, labels, or markers that help organize and describe the data. It lies between structured and unstructured data.

- Does not have a fixed schema.
- Contains metadata or tags.
- Flexible structure.
- Easier to process than unstructured data.

Example (XML)

```
<Student>
  <Name>Amit</Name>
  <Marks>85</Marks>
</Student>
```

Advantages

- Flexible and adaptable.
- Easier to exchange between systems.
- Can store complex information.

Examples

- XML documents
- JSON files
- HTML pages
- Emails

Limitations

- More difficult to analyze than structured data.

- Requires special processing techniques.

3. **Unstructured Data:** refers to data that does not have a predefined format, schema, or organization. It cannot be easily stored in traditional relational databases. This is the most common type of data generated today.

- No fixed structure.
- Difficult to store and analyze using conventional methods.
- Exists in multiple formats.
- Requires advanced Big Data tools for processing.

- Documents
- Chat messages

Advantages

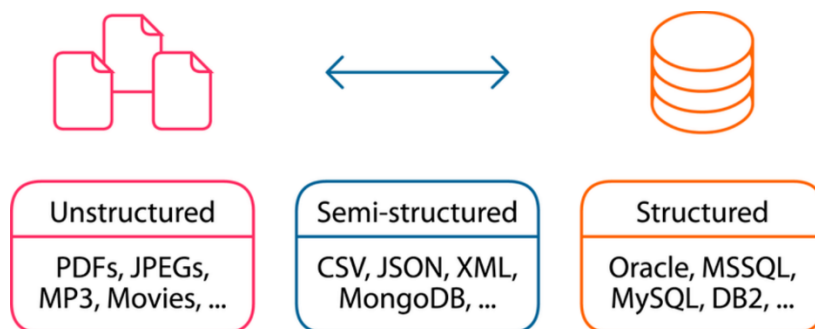
- Contains rich and valuable information.
- Provides deeper insights into user behavior and preferences.

Examples

- Images
- Videos
- Audio files
- Social media posts
- Emails

Limitations

- Difficult to manage.
- Requires large storage capacity.
- Complex analysis techniques are needed.



Importance of Unstructured Data

- **Largest Source of Data:** More than 80% of the world's data is unstructured, including social media content, images, videos, emails, and documents.
- **Better Customer Insights:** Organizations analyze reviews, comments, and social media posts to understand customer preferences and behavior.
- **Improved Decision Making:** Analysis of unstructured data helps businesses make informed decisions based on real-world trends and information.
- **Supports Advanced Analytics:** Technologies such as AI, Machine Learning (ML), and Natural Language Processing (NLP) rely heavily on unstructured data.
- **Healthcare Applications:** Medical images, doctor's notes, and patient records provide valuable information for diagnosis and treatment.
- **Fraud Detection and Security:** Emails, messages, and user activities can be analyzed to identify fraud and security threats.
- **Business Innovation:** Unstructured data helps organizations discover new opportunities, improve products, and enhance customer experiences.

Comparison of Data Types

Feature	Structured Data	Semi-Structured Data	Unstructured Data
Schema	Fixed	Partial	None
Organization	Rows and Columns	Tags and Metadata	No predefined structure
Storage	Relational Databases	XML, JSON Databases	Hadoop, NoSQL, Data Lakes
Processing	Easy	Moderate	Complex
Examples	Student Records, Banking Data	XML, JSON, Emails	Images, Videos, Social Media Posts

Applications of Big Data

Industry Examples of Big Data

1. **Banking:** Big Data helps banks detect fraud, assess credit risk, personalize financial services, and improve customer relationship management.
2. **Retail:** Retail companies analyze customer purchasing patterns, manage inventory, forecast demand, and provide personalized product recommendations.
3. **Telecommunications:** Telecom companies use Big Data to monitor network performance, predict equipment failures, reduce customer churn, and improve service quality.
4. **Transportation:** Transportation organizations analyze traffic patterns, optimize routes, manage fleets, and improve passenger safety and convenience.
5. **Manufacturing:** Manufacturers use Big Data for predictive maintenance, quality control, supply chain optimization, and production planning.

Web Analytics

Web Analytics is the process of collecting, measuring, analyzing, and reporting website data to understand user behavior and improve website performance. **Components**

- **Data Collection:** Gathering information about website visitors.
- **Data Measurement:** Measuring user activities and interactions.
- **Data Analysis:** Identifying patterns and trends.
- **Reporting:** Presenting insights through reports and dashboards.
- **Conversion Rate:** Percentage of visitors who complete a desired action such as registration or purchase.

Benefits

- **Improved User Experience:** Helps identify user preferences and website issues.
- **Better Marketing Decisions:** Measures effectiveness of marketing campaigns.
- **Increased Sales:** Optimizes website performance and conversions.
- **Customer Understanding:** Provides insights into visitor behavior.

Metrics

- **Page Views:** Total number of times web pages are viewed by users.
- **Visitors:** Number of unique users visiting the website.
- **Bounce Rate:** Percentage of visitors who leave the website after viewing only one page.

Big Data in Marketing

1. Big Data enables organizations to understand customers, predict behavior, and deliver personalized marketing strategies.
2. **Customer Segmentation:** Divides customers into groups based on demographics, interests, purchasing behavior, or location.
3. **Customer Behavior Analysis:** Analyzes customer activities and purchasing patterns to understand preferences and future needs.
4. **Personalized Marketing:** Delivers customized advertisements, offers, and recommendations based on customer interests.
5. **Recommendation Systems:** Suggests products or services based on previous purchases and browsing history.
6. **Sentiment Analysis:** Analyzes customer opinions from reviews, comments, and social media posts to determine positive or negative sentiments.
7. **Targeted Advertising:** Displays advertisements to specific customer groups that are most likely to be interested in the product.

Fraud Detection using Big Data

Types of Fraud

1. **Banking Fraud:** Illegal activities such as account manipulation, identity theft, and unauthorized transactions.
2. **Credit Card Fraud:** Unauthorized use of credit card information for financial gain.
3. **Insurance Fraud:** Submission of false insurance claims to obtain benefits illegally.
4. **Online Transaction Fraud:** Fraudulent activities involving online purchases, payments, or digital transactions.

Fraud Detection Process

1. Collect transaction and customer data.
2. Analyze patterns and behaviors.
3. Identify suspicious activities.
4. Generate alerts for investigation.
5. Take preventive actions.

Benefits

- **Real-Time Detection:** Identifies fraud immediately.
- **Reduced Financial Losses:** Prevents fraudulent transactions.
- **Improved Security:** Enhances protection of customer information.
- **Better Compliance:** Supports regulatory requirements.

Risk Analytics using Big Data

Risk Management: Risk management involves identifying, assessing, and controlling potential threats that may affect an organization.

Types of Risks

1. **Financial Risk:** Risk of financial loss due to market fluctuations, investments, or economic conditions.

2. **Operational Risk:** Risk arising from internal failures, system breakdowns, or human errors.
3. **Market Risk:** Risk caused by changes in market prices, interest rates, or exchange rates.

Role of Big Data in Risk Analysis

- **Risk Identification:** Detects potential risks using historical and real-time data.

- **Predictive Analysis:** Forecasts future risks and trends.
- **Better Decision Making:** Supports informed risk management strategies.
- **Continuous Monitoring:** Tracks risks in real time.

Credit Risk Management

1. **Credit Risk** Credit risk is the possibility that a borrower may fail to repay a loan or meet financial obligations.
2. **Credit Scoring:** A method of evaluating a customer's creditworthiness based on financial history and behavior.
3. **Loan Approval Analysis:** Big Data helps financial institutions assess loan applications by analyzing customer profiles and repayment capacity.

Data Sources Used

- Credit history
- Transaction records
- Income details
- Loan repayment records
- Customer behavior data

Benefits

- **Better Loan Decisions:** Improves approval accuracy.
- **Reduced Default Risk:** Identifies high-risk borrowers.
- **Faster Processing:** Automates risk assessment.
- **Improved Profitability:** Reduces financial losses.

Big Data and Algorithmic Trading

1. **Algorithmic Trading:** Algorithmic trading uses computer programs to automatically execute financial transactions based on predefined rules.
2. **Trading Algorithms:** Algorithms analyze market data and make trading decisions without human intervention.
3. **Market Data Analysis:** Big Data systems process large volumes of financial data, news feeds, and market trends to identify opportunities.
4. **Real-Time Decision Making:** Trading systems make instant decisions based on continuously changing market conditions.

Advantages

- **High Speed:** Executes trades within milliseconds.
- **Reduced Human Error:** Minimizes emotional decision-making.
- **Better Accuracy:** Uses data-driven strategies.
- **Efficient Market Analysis:** Processes huge datasets quickly.

Big Data in Healthcare

1. **Healthcare Data Sources:** Healthcare data is collected from hospitals, clinics, laboratories, wearable devices, and patient records.
2. **Electronic Health Records (EHR):** Digital versions of patient medical histories, diagnoses, treatments, and prescriptions.
3. **Patient Monitoring:** Wearable devices and sensors continuously monitor patient health conditions in real time.
4. **Disease Prediction:** Big Data helps predict diseases by analyzing medical

records, lifestyle data, and genetic information.

5. **Hospital Management:** Supports resource allocation, staff scheduling, inventory management, and operational efficiency.

Big Data in Medicine

1. **Personalized Medicine:** Medical treatments are customized according to an individual's genetic and health profile.
2. **Genomic Analysis:** Analysis of DNA and genetic information to identify disease risks and treatment options.

3. **Drug Discovery:** Big Data accelerates the identification and development of new medicines.

4. **Clinical Research:** Researchers analyze large datasets to evaluate treatments and improve medical knowledge.

5. **Precision Medicine:** A medical approach that provides targeted treatments based on patient-specific characteristics.

Advertising and Big Data

1. **Digital Advertising:** Uses online platforms and customer data to deliver advertisements through websites, social media, and mobile applications.
2. **Customer Profiling:** Creates detailed profiles of customers based on demographics, interests, and behavior.
3. **Ad Targeting:** Displays advertisements to users who are most likely to respond positively.
4. **Behavioral Advertising:** Uses browsing history, search activity, and online behavior to personalize advertisements.
5. **Ad Performance Analysis:** Measures advertisement effectiveness using metrics such as clicks, impressions, and conversion rates.

Big Data Technologies

Big Data technologies are tools and frameworks used to store, process, manage, and analyze large volumes of structured, semi-structured, and unstructured data. These technologies help organizations handle data that cannot be efficiently processed using traditional systems.

- 1) **Hadoop** An open-source framework that enables the distributed storage and processing of large datasets across multiple computers. It consists mainly of HDFS (storage) and MapReduce (processing).

Features

- Distributed data storage.
- Fault tolerance.
- Scalability.
- Cost-effective processing of large datasets.

Applications

- Data warehousing.

- Log analysis.
- Large-scale data processing.

- 2) **NoSQL Databases:** Non-relational databases designed to store and manage large volumes of diverse and rapidly changing data. **Features**

- Flexible schema.
- High scalability.
- Fast data access.
- Support for unstructured data.

Examples

- MongoDB
- Cassandra

- HBase
- CouchDB

Applications

- Social media platforms.
- Real-time web applications.
- Big Data analytics.

3) **Apache Spark:** A fast, open-source data processing framework that performs large-scale data analytics using in-memory computing. **Features**

- High-speed processing.
- Real-time analytics.
- In-memory computation.
- Support for machine learning and streaming.

Applications

- Data analytics.
- Machine learning.
- Real-time data processing.

4) **Hive:** A data warehouse tool built on Hadoop that allows users to query and analyze large datasets using Hive Query Language (HiveQL), which is similar to SQL.

Features of Hive

- SQL-like query language.
- Suitable for data warehousing.
- Easy data summarization.
- Runs on Hadoop.

Applications

- Data analysis.
- Report generation.
- Business intelligence.

5) **Pig:** A high-level data processing platform that uses a scripting language called Pig Latin to analyze large datasets on Hadoop.

Features

- Simple scripting language.
- Supports complex data processing.
- Reduces coding effort.
- Works on Hadoop.

Applications

- Data transformation.
- ETL (Extract, Transform, Load) operations.
- Data analysis.

6) **Kafka:** A distributed messaging and streaming platform used to collect, store, and process real-time data streams.

Features

- Real-time data streaming.
- High throughput.
- Fault tolerance.
- Scalable architecture.

Applications

- Log monitoring.
- Event tracking.
- Real-time analytics.

Technology	Main Purpose
Hadoop	Distributed storage and processing
NoSQL Databases	Storage of non-relational data
Apache Spark	Fast data processing and analytics
Hive	SQL-based data warehousing
Pig	Data processing using scripts
Kafka	Real-time data streaming

Unit 1: Big Data and Hadoop Ecosystem

1. Introduction to Big Data

- What is Big Data?
 - Characteristics of Big Data (Vs)
 - Volume
 - Velocity
 - Variety
 - Veracity
 - Value
 - Why Big Data?
-

2. Evolution of Big Data

- Convergence of Key Trends
 - Internet Growth
 - Social Media
 - Cloud Computing
 - Mobile Computing
 - IoT
 - Cheap Storage
-

3. Types of Data

- Structured Data
 - Semi-Structured Data
 - Unstructured Data
 - Importance of Unstructured Data
-

4. Industry Applications of Big Data

- Industry Examples of Big Data
 - Banking
 - Retail
 - Telecommunications
 - Transportation
 - Manufacturing
-

5. Web Analytics

- Definition
 - Components
 - Metrics
 - Page Views
 - Visitors
 - Bounce Rate
 - Conversion Rate
 - Benefits
-

6. Big Data in Marketing

- Customer Segmentation
- Customer Behavior Analysis

- Personalized Marketing
 - Recommendation Systems
 - Sentiment Analysis
 - Targeted Advertising
-

7. Fraud Detection using Big Data

- Types of Fraud
 - Banking Fraud
 - Credit Card Fraud
 - Insurance Fraud
 - Online Transaction Fraud
 - Fraud Detection Process
 - Benefits
-

8. Risk Analytics using Big Data

- Risk Management
 - Types of Risks
 - Financial Risk
 - Operational Risk
 - Market Risk
 - Role of Big Data in Risk Analysis
-

9. Credit Risk Management

- Credit Risk
 - Credit Scoring
 - Loan Approval Analysis
 - Data Sources Used
 - Benefits
-

10. Big Data and Algorithmic Trading

- Algorithmic Trading
 - Trading Algorithms
 - Market Data Analysis
 - Real-Time Decision Making
 - Advantages
-

11. Big Data in Healthcare

- Healthcare Data Sources
 - Electronic Health Records (EHR)
 - Patient Monitoring
 - Disease Prediction
 - Hospital Management
-

12. Big Data in Medicine

- Personalized Medicine

- Genomic Analysis
 - Drug Discovery
 - Clinical Research
 - Precision Medicine
-

13. Advertising and Big Data

- Digital Advertising
 - Customer Profiling
 - Ad Targeting
 - Behavioral Advertising
 - Ad Performance Analysis
-

14. Big Data Technologies

- Hadoop
 - NoSQL Databases
 - Apache Spark
 - Hive
 - Pig
 - Kafka
 - Other Big Data Tools
-

15. Introduction to Hadoop

- What is Hadoop?
- Need for Hadoop
- Hadoop Features
- Hadoop Ecosystem
- Advantages of Hadoop

Hadoop Core Components

- HDFS
 - MapReduce
 - YARN
 - Hadoop Common
-

16. Open Source Technologies

- Meaning of Open Source
 - Benefits
 - Examples
 - Hadoop
 - Hive
 - Pig
 - Spark
 - Cassandra
-

17. Cloud and Big Data

- Cloud Computing Basics

- Cloud Storage
- Cloud Analytics
- Benefits of Cloud for Big Data
- Big Data as a Service (BDaaS)

18. Mobile Business Intelligence (Mobile BI)

- Definition
- Mobile Analytics
- Real-Time Reporting
- Mobile Dashboards
- Advantages

19. Crowdsourcing Analytics

- Definition
- Data Collection from Large Groups
- Crowdsourced Data Sources
- Applications
- Benefits and Challenges

20. Inter and Trans Firewall Analytics

Inter-Firewall Analytics

- Analytics Across Multiple Networks
- Security Monitoring
- Data Sharing Between Organizations

Trans-Firewall Analytics

- Analytics Beyond Organizational Boundaries
- Cross-Enterprise Data Analysis
- Security and Privacy Concerns

Exam Priority (Most Important)

Very Important:

1. Big Data & 5Vs
2. Why Big Data
3. Structured vs Semi-Structured vs Unstructured Data
4. Hadoop Introduction
5. Hadoop Components (HDFS, MapReduce, YARN)
6. Big Data Technologies
7. Big Data in Healthcare
8. Big Data in Marketing
9. Web Analytics
10. Fraud Detection & Risk Analytics

Moderately

11. Credit Risk
12. Algorithmic

Important:
Management
Trading

13. Advertising and Big Data
14. Cloud and Big Data
- Usually Short Notes:
15. Mobile BI
16. Crowdsourcing Analytics
17. Open Source Technologies
18. Inter and Trans Firewall Analytics

Module 1.2: Introduction to Hadoop Ecosystem

Contents

Hadoop	1
Hadoop Ecosystem	2
Open-Source Technologies.....	4
Cloud and Big Data.....	5
Cloud Storage	6
Cloud Analytics.....	6
Big Data as a Service (BDaaS)	6
Mobile Business Intelligence (Mobile BI)	7
Mobile Analytics.....	7
Real-Time Reporting	7
Mobile Dashboards	8
Crowdsourcing Analytics.....	8
Data Collection from Large Groups	8
Inter-Firewall Analytics	10
Security Monitoring.....	10
Data Sharing Between Organizations	10
Trans-Firewall Analytics	10
Cross-Enterprise Data Analysis.....	10
Security and Privacy Concerns	11

Hadoop

Hadoop: Hadoop is an open-source framework developed by the Apache Software Foundation for storing and processing large volumes of data across multiple computers in a distributed environment. It is designed to handle structured, semi-structured, and unstructured data efficiently using clusters of commodity hardware.

Need for Hadoop

Traditional database systems face limitations when dealing with massive amounts of data. Hadoop was developed to overcome these challenges.

- **Handling Big Data:** Processes huge volumes of data that traditional systems cannot handle efficiently.
- **Scalability:** New nodes can be added easily as data grows.
- **Cost Effectiveness:** Uses low-cost commodity hardware instead of expensive servers.
- **Fault Tolerance:** Continues operation even if some nodes fail.

- **Distributed Processing:** Processes data in parallel across multiple machines, improving performance.

Hadoop Features

- **Distributed Storage:** Stores data across multiple machines.
- **Distributed Processing:** Processes data simultaneously on different nodes.
- **Large-Scale Data Processing:** Handles terabytes and petabytes of data efficiently.
- **Scalability:** Supports growth from a few nodes to thousands of nodes.
- **High Processing Speed:** Uses parallel processing techniques.

- **Open Source:** Freely available and supported by a large community.
- **Fault Tolerance:** Automatically replicates data to prevent loss.
- **High Availability:** Ensures continuous access to data.
- **Flexibility:** Supports structured, semi-structured, and unstructured data.
- **Cost Efficiency:** Uses inexpensive hardware for storage and processing.

Hadoop Ecosystem

Hadoop Ecosystem: A collection of tools and technologies that work together with Hadoop to store, process, analyze, and manage Big Data.

1. **HDFS (Hadoop Distributed File System):** is the primary storage system of Hadoop. It is designed to store very large files across multiple machines in a distributed environment. Instead of storing a file on a single computer, HDFS divides the file into blocks and stores them on different nodes in a cluster.

Working: When a file is uploaded to HDFS, it is broken into smaller blocks (typically 128 MB or 256 MB). These blocks are distributed among different DataNodes. To ensure reliability, each block is replicated and stored on multiple nodes. If one node fails, the data can still be accessed from another node.

Features

- Distributed storage of large files.
- Automatic data replication.
- Fault tolerance.
- High scalability.
- High throughput access to data.

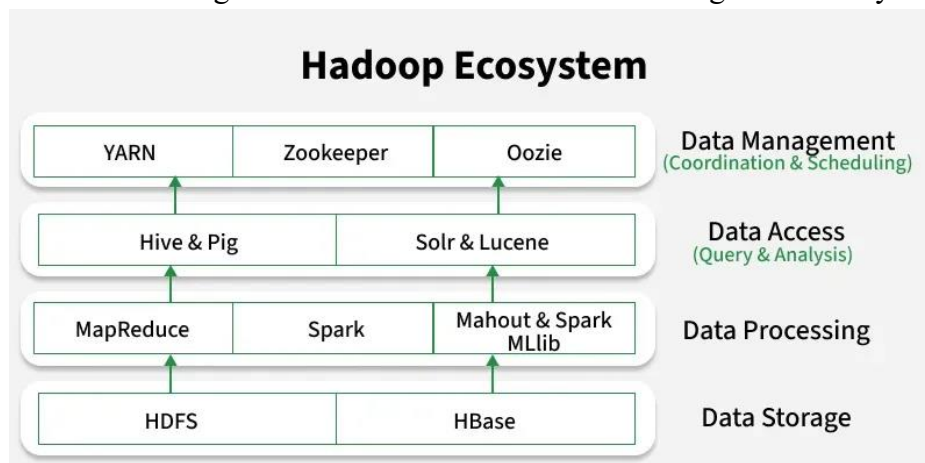
Advantages

- Reliable data storage.

- Supports petabytes of data.
- Fault recovery through replication.
- Cost-effective storage solution.

Applications

- Data warehousing.
- Log storage.
- Social media data storage.
- Large-scale analytics.



2. **MapReduce:** is the data processing framework of Hadoop. It processes large datasets in parallel by dividing the work among multiple machines and combining the results to generate final output.

Working: MapReduce consists of two phases:

- **Map Phase:** The input data is divided into smaller chunks and processed independently. Each mapper generates intermediate key-value pairs.
- **Reduce Phase:** The reducer collects the intermediate key-value pairs, groups similar keys together, and produces the final output.

Features

- Parallel processing.
- Distributed computation.
- Automatic fault handling.
- Scalable processing framework.

Advantages

- Processes massive datasets efficiently.

- Reduces execution time.
- Supports distributed computing.
- Improves system performance.

Applications

- Log analysis.
- Data mining.
- Search engine indexing.
- Business analytics.

3. **YARN (Yet Another Resource Negotiator):** is the resource management and job scheduling framework of Hadoop. It manages cluster resources and allocates them to different applications running in the Hadoop environment.

Working: When a job is submitted, YARN determines which resources are required and allocates CPU, memory, and storage resources accordingly. It continuously monitors running applications and ensures efficient resource utilization.

Main Components of YARN

- **Resource Manager:** Manages and allocates cluster resources.
- **Node Manager:** Runs on each node and monitors resource usage.
- **Application Master:** Manages the execution of a particular application.
- **Containers:** Units where application tasks are executed.

Features

- Efficient resource management.
- Multi-user support.
- Job scheduling.
- Cluster monitoring.

Advantages

- Better cluster utilization.
- Improved scalability.

- Supports multiple processing frameworks.
- Efficient workload management.

Applications

- Resource allocation.
- Cluster management.
- Job scheduling.

4. **Hive:** is an open-source data warehouse system built on Hadoop that allows users to query large datasets using HiveQL, a language similar to SQL. It enables users who are familiar with SQL to analyze Big Data without writing complex MapReduce programs.

Working: Hive converts HiveQL queries into MapReduce, Tez, or Spark jobs and executes them on Hadoop.

Features

- SQL-like query language.
- Data summarization.
- Query optimization.
- Data warehousing support.

Advantages

- Easy to learn for SQL users.
- Simplifies data analysis.
- Supports large datasets.

- Reduces programming complexity.

Applications

- Business intelligence.
- Report generation.
- Data analysis.
- Data warehousing.

Example

SELECT * FROM Employees;

5. **Pig:** is an open-source data processing platform used for analysing large datasets on Hadoop. It uses a scripting language called Pig Latin. Pig simplifies complex data processing tasks that would otherwise require lengthy MapReduce code.

Working: Users write Pig Latin scripts which are automatically converted into MapReduce jobs and executed on Hadoop.

Features

- High-level scripting language.
- Supports structured and unstructured data.
- Reduces coding effort.
- Easy data transformation.

Advantages

- Faster development.
- Less coding than MapReduce.

- Suitable for ETL operations.
- Easy debugging.

Applications

- Data transformation.
- Data cleansing.
- ETL processes.
- Data analysis.

Example

A = LOAD 'employee.txt';

6. **HBase:** is a distributed NoSQL database built on top of HDFS. It is designed for storing huge amounts of structured and semi-structured data while providing real-time read and write access.

Features

- Column-oriented database.
- Distributed architecture.
- Real-time access.
- High scalability.

Advantages

- Fast data retrieval.
-

- Handles billions of records.
- Fault tolerant.
- Supports large-scale applications.

Applications

- Social media applications.
- Online transaction systems.
- Real-time analytics.

7. **Apache Spark:** is an open-source distributed computing framework that performs high-speed data processing in memory instead of repeatedly reading from disk. It is considered much faster than traditional MapReduce processing.

Working: Spark loads data into memory and performs computations directly in RAM, reducing disk access and improving processing speed.

Features

- In-memory computing.
- Real-time processing.
- Machine learning support.
- Stream processing.
- Graph processing.

Advantages

- Faster than MapReduce.

- Supports batch and real-time analytics.
- Easy integration with Hadoop.
- Supports advanced analytics.

Applications

- Machine learning.
- Data analytics.
- Real-time processing.
- Recommendation systems.

Open-Source Technologies

Open Source Technology refers to software whose source code is freely available to the public. Users can view, modify, distribute, and improve the software according to their requirements. Open-source software is usually developed collaboratively by a community of developers and organizations.

- Source code is publicly accessible.
- Users can modify and customize the software.
- No licensing fees in most cases.
- Community-driven development.
- Continuous improvement through collaboration.

Benefits of Open Source Technologies

- 1. Cost Effectiveness:** Open-source software is generally available free of cost, reducing software licensing expenses.
- 2. Flexibility and Customization:** Users can modify the source code according to their specific requirements.
- 3. Transparency:** The source code is publicly available, allowing users to inspect and verify the software.
- 4. Community Support:** Large developer communities continuously contribute updates, bug fixes, and improvements.
- 5. Reliability:** Continuous testing and contributions make open-source software stable and dependable.
- 6. Security:** Vulnerabilities can be identified and fixed quickly due to public code review.
- 7. Scalability:** Open-source technologies can easily handle increasing amounts of data and workloads.
- 8. Vendor Independence:** Organizations are not locked into a single vendor and have complete control over the software.
- 9. Faster Innovation:** Continuous contributions from developers worldwide help introduce new features rapidly.
- 10. Easy Integration:** Most open-source tools can be integrated easily with other technologies and platforms.

Examples of Open-Source Technologies: Hadoop, Hive, Pig, Apache Spark

Comparison of Open-Source Big Data Technologies

Technology	Primary Purpose
Hadoop	Distributed storage and processing
Hive	SQL-based data analysis
Pig	Data processing using scripts
Spark	Fast in-memory data processing
Cassandra	Distributed NoSQL database

Cloud and Big Data

Cloud Computing is the delivery of computing resources such as servers, storage, databases, networking, and software over the Internet on a pay-as-you-use basis.

- On-demand access to resources.
- Scalability and flexibility.
- Resource sharing.
- Remote accessibility.
- Pay-per-use pricing model.

Service Models

- **IaaS (Infrastructure as a Service):** Provides virtual machines, storage, and networking resources.
- **PaaS (Platform as a Service):** Provides a platform for developing and deploying applications.
- **SaaS (Software as a Service):** Provides software applications over the Internet.

Cloud Storage

Cloud Storage is a service that allows data to be stored, managed, and accessed over the Internet instead of local storage devices. **Features**

- Large storage capacity.
- Automatic backup and recovery.
- Data accessibility from anywhere.
- High availability and reliability.
- Easy scalability.
- Microsoft OneDrive
- Amazon S3

Examples

- Google Drive
- Dropbox

Importance in Big Data: Cloud storage enables organizations to store massive volumes of structured, semi-structured, and unstructured data without investing in expensive hardware.

Cloud Analytics

Cloud Analytics refers to the use of cloud-based tools and services to process, analyze, and visualize large volumes of data. **Functions**

- Data collection.
- Data processing.
- Data analysis.
- Reporting and visualization.
- Predictive analytics.
- Real-time analytics.
- Reduced infrastructure costs.
- Easy integration with Big Data tools.

Examples

- Google BigQuery
- Amazon Redshift
- Microsoft Azure Synapse Analytics

Advantages

- Faster data processing.

Benefits of Cloud for Big Data

1. **Scalability:** Cloud platforms can easily increase or decrease resources based on data requirements.
2. **Cost Efficiency:** Organizations pay only for the resources they use, reducing infrastructure costs.
3. **High Storage Capacity:** Cloud environments can store enormous amounts of Big Data.
4. **Faster Processing:** Powerful cloud resources enable rapid processing of large datasets.
5. **Flexibility:** Resources can be allocated dynamically according to workload demands.
6. **Accessibility:** Data and applications can be accessed from anywhere through the Internet.
7. **Reliability:** Cloud providers offer backup, disaster recovery, and fault-tolerant systems.
8. **Real-Time Analytics:** Cloud platforms support real-time data processing and decision-making.
9. **Easy Deployment:** Big Data applications can be deployed quickly without purchasing hardware.
10. **Enhanced Collaboration:** Multiple users can access and analyze data simultaneously.

Big Data as a Service (BDaaS)

Big Data as a Service (BDaaS) is a cloud-based service model that provides Big Data storage, processing, analytics, and management capabilities over the Internet.

Working: Organizations upload data to a cloud platform where the service provider manages infrastructure, storage, processing, and analytics tools.

Features

- Cloud-based Big Data processing.
- On-demand analytics services.
- Reduced infrastructure management.
- Pay-as-you-go pricing.
- Scalable resources.

Advantages

1. **Lower Cost:** Eliminates the need for expensive hardware and maintenance.
2. **Scalability:** Resources can be increased or decreased as needed.
3. **Faster Implementation:** Big Data solutions can be deployed quickly.
4. **Reduced Complexity:** Service providers handle infrastructure management.

5. **Improved Accessibility:** Data and analytics tools are accessible from anywhere.

Examples of BDaaS Platforms

- Amazon Web Services (AWS)
- Google Cloud Platform (GCP)
- Microsoft Azure
- IBM Cloud

Relationship Between Cloud and Big Data

Cloud computing provides the infrastructure, storage, and processing power required for Big Data applications. Big Data generates enormous volumes of information, while cloud platforms offer scalable and cost-effective solutions for storing, processing, and analyzing that data. Together, they enable organizations to extract valuable insights and make data-driven decisions efficiently.

Mobile Business Intelligence (Mobile BI)

Mobile Business Intelligence (Mobile BI) refers to the delivery of Business Intelligence (BI) tools, reports, dashboards, and analytics on mobile devices such as smartphones and tablets, enabling users to access business information anytime and anywhere.

- Access to business data through mobile devices.
- Real-time monitoring of business performance.
- Interactive reports and dashboards.
- Supports decision-making on the move.

Mobile Analytics

Mobile Analytics is the process of collecting, measuring, and analyzing business data through mobile applications and devices to gain insights and support decision-making.

Uses

- Tracking sales performance.
- Monitoring customer behavior.
- Analyzing business trends.
- Measuring application usage.

Benefits

- Quick access to data insights.
- Better understanding of customer activities.
- Faster business decisions.

Real-Time Reporting

Real-Time Reporting provides up-to-date business information immediately after data is generated, allowing organizations to respond quickly to changing situations.

Features

- Instant data updates.
- Live monitoring of operations.
- Immediate identification of issues.
- Supports rapid decision-making.

Examples

- Live sales reports.
- Stock market updates.
- Website traffic monitoring.
- Fraud detection alerts.

Mobile Dashboards

Mobile Dashboards are visual interfaces designed for mobile devices that display key business metrics, reports, and performance indicators in an easy-to-understand format.

Components

- Charts and graphs.
- Performance indicators (KPIs).
- Reports and summaries.
- Alerts and notifications.

Benefits

- Easy visualization of data.
- Quick performance monitoring.
- Improved accessibility.
- User-friendly interface

Advantages of Mobile BI

- 1. Anywhere Access:** Users can access business information from any location using mobile devices.
- 2. Real-Time Decision Making:** Provides instant access to current business data for faster decisions.
- 3. Increased Productivity:** Employees can monitor and manage business activities while away from the office.
- 4. Improved Collaboration:** Teams can share reports and insights quickly through mobile platforms.
- 5. Better Customer Service:** Real-time information helps organizations respond to customer needs more effectively.
- 6. Faster Reporting:** Business reports can be viewed instantly without waiting for desktop access.
- 7. Enhanced Monitoring:** Managers can continuously track business performance and key metrics.
- 8. Cost Efficiency:** Reduces dependence on physical infrastructure and office-based reporting systems.

Crowdsourcing Analytics

Crowdsourcing Analytics is the process of collecting, analyzing, and utilizing data contributed by a large number of people through online platforms, mobile applications, social media, and other digital channels. **Key Features**

- Data is gathered from a large group of individuals.
- Uses internet-based platforms for participation.
- Provides diverse and real-time information.
- Supports large-scale data collection.

Data Collection from Large Groups

Data Collection from Large Groups involves gathering information, opinions, feedback, or observations from thousands or millions of participants.

Methods

- Online surveys.
- Social media contributions.
- Mobile applications.
- User reviews and ratings.
- Crowdsourcing platforms.

Importance

- Generates large datasets quickly.
- Provides diverse viewpoints.
- Reduces data collection costs.
- Improves data availability.

Crowdsourced Data Sources

- **Social Media:** Platforms like Facebook, X (Twitter), and Instagram generate user-created content.
- **Online Reviews:** Customer reviews on e-commerce and service websites provide valuable insights.
- **Mobile Applications:** Apps collect user feedback, location data, and activity information.
- **Crowdsourcing Platforms:** Websites specifically designed for collecting information from large groups.
- **Community Forums:** Discussion boards and online communities contribute opinions and experiences.
- **Wikipedia:** Information is created and maintained by a large community of contributors.

Applications of Crowdsourcing Analytics

1. **Market Research:** Organizations analyze customer opinions and preferences to improve products and services.
2. **Sentiment Analysis:** Businesses study social media content to understand public attitudes toward brands and products.
3. **Traffic Monitoring:** Navigation applications collect user-generated location data to provide real-time traffic updates.
4. **Disaster Management:** Information from citizens helps authorities respond quickly during emergencies.
5. **Product Development:** Customer suggestions and feedback help organizations develop better products.
6. **Healthcare Research:** Large groups contribute health-related information for medical studies and disease tracking.
7. **Content Creation:** Crowdsourced information helps build knowledge bases and online communities.

Benefits

1. **Large Data Volume:** Enables collection of massive amounts of data from numerous participants.
2. **Cost Effective:** Reduces the cost of traditional data collection methods.
3. **Real-Time Information:** Data can be collected and analyzed quickly.
4. **Diverse Perspectives:** Information comes from people with different backgrounds and experiences.
5. **Scalability:** Easily gathers data from a large population.
6. **Faster Decision Making:** Organizations gain timely insights for better decisions.

Challenges

1. **Data Quality Issues:** User-contributed data may be inaccurate or incomplete.
2. **Data Reliability:** Not all contributors provide trustworthy information.
3. **Privacy Concerns:** Collection of user data may raise privacy and security issues.
4. **Data Validation:** Large datasets require verification before analysis.
5. **Bias in Contributions:** Data may not represent all population groups equally.
6. **Security Risks:** Crowdsourced platforms can be vulnerable to false or malicious information.

Inter-Firewall Analytics

Inter-Firewall Analytics/Analytics Across Multiple Networks refers to the process of collecting and analyzing data from multiple networks protected by different firewalls to gain a broader view of operations, performance, and security. **Importance**

- Provides a unified view of distributed networks.
- Helps detect network-wide issues.
- Improves coordination across multiple locations.
- Supports enterprise-wide decision making.

Security Monitoring

Security Monitoring involves continuously analyzing network traffic, logs, and security events across multiple firewall-protected networks to identify threats and suspicious activities.

Benefits

- Early detection of cyber attacks.
- Continuous monitoring of network activities.
- Improved incident response.
- Enhanced network security.

Examples

- Intrusion detection.
- Malware monitoring.
- Unauthorized access detection.
- Threat intelligence analysis

Data Sharing Between Organizations

Data Sharing Between Organizations allows multiple organizations to exchange selected data securely for collaborative analysis while maintaining network protection through firewalls.

Applications

- Banking and financial fraud detection.
- Healthcare research collaborations.
- Supply chain management.
- Government security operations.

Benefits

- Better insights through combined data.
- Improved collaboration.
- Enhanced decision-making.
- More effective threat detection.

Trans-Firewall Analytics

Trans-Firewall Analytics/Analytics Beyond Organizational Boundaries refers to analyzing data that originates from different organizations, departments, or external sources beyond a single firewall-protected environment.

Importance

- Provides broader business insights.
- Supports large-scale analytics.
- Enables collaborative research and innovation.
- Improves strategic planning.

Cross-Enterprise Data Analysis

Cross-Enterprise Data Analysis involves combining and analyzing data from multiple organizations to identify patterns, trends, and opportunities that may not be visible within a single organization.

Applications

- Industry benchmarking.
- Fraud detection across institutions.
- Market research.

- Healthcare analytics.
- Supply chain optimization.

Benefits

- More comprehensive analysis.

- Improved predictive capabilities.
- Better risk assessment.
- Enhanced business intelligence.

Security and Privacy Concerns

Security and Privacy Concerns arise when data is shared and analyzed across organizational boundaries, requiring strict measures to protect sensitive information. **Major Concerns**

- 1. Data Privacy:** Personal and confidential information must be protected from unauthorized access.
- 2. Data Security:** Shared data must be secured during storage and transmission.
- 3. Regulatory Compliance:** Organizations must comply with data protection laws and regulations.
- 4. Access Control:** Only authorized users should access shared data.

- 5. Data Breaches:** Shared environments may increase the risk of cyber attacks and information leakage.

Solutions

- Data encryption.
- Authentication and authorization.
- Secure communication channels.
- Data anonymization.
- Regular security audits.

Difference Between Inter-Firewall and Trans-Firewall Analytics

Inter-Firewall Analytics	Trans-Firewall Analytics
Operates across multiple firewall-protected networks.	Operates beyond organizational boundaries.
Focuses on network-level monitoring and analysis.	Focuses on cross-enterprise data analysis.
Usually within related organizations or departments.	Often involves multiple independent organizations.
Primarily used for security and operational monitoring.	Primarily used for collaboration and large-scale analytics.

Module 2.1: NoSQL Databased Models

Contents

NoSQL Databases	1
Need for NoSQL Databases	2
Characteristics of NoSQL Databases	2
Advantages of NoSQL Databases	2
Aggregate Data Models	3
Importance of Aggregate Data Models	3
Benefits of Aggregate-Oriented Design	3
Aggregates	4
Aggregate Boundaries	4
Uses of Aggregates	4
Advantages of Aggregates	5
Key-Value Data Model	5
Document Data Model	6
Relationships in NoSQL Databases	7
Graph Databases	8
Nodes	8
Edges	8
Properties	9
Schemeless Databases	10
Materialized Views	11
Working of Materialized Views	11

NoSQL Databases

NoSQL (Not Only SQL) refers to a category of database management systems that store and manage data in formats other than traditional relational tables. NoSQL databases are designed to handle large volumes of structured, semi-structured, and unstructured data while providing high scalability and performance. **Key Points**

- NoSQL stands for "**Not Only SQL.**"
- Uses flexible data models instead of fixed table structures.
- Supports distributed storage across multiple servers.
- Suitable for Big Data and real-time web applications.
- Provides horizontal scalability.
- Redis
- HBase
- CouchDB

Examples

- MongoDB
- Cassandra

Applications Requiring NoSQL

- Social media platforms
- E-commerce websites
- IoT systems
- Cloud applications
- Real-time analytics systems

Need for NoSQL Databases

Traditional relational databases face challenges when dealing with massive data volumes, high-speed transactions, and rapidly changing data structures. NoSQL databases were developed to overcome these limitations. **Reasons for the Need of NoSQL**

1. Handling Big Data

- Modern applications generate huge amounts of data.
- NoSQL databases efficiently store and process large datasets.

2. High Scalability

- Data can be distributed across multiple servers.
- Supports horizontal scaling by adding new machines.

3. Flexible Data Structure

- No fixed schema is required.

- Easy to modify data formats without redesigning tables.

4. High Performance

- Provides faster read and write operations.
- Suitable for real-time applications.

5. Distributed Environment Support

- Designed for cloud and distributed computing systems.
- Ensures availability and fault tolerance.

Characteristics of NoSQL Databases

1. Schema-less Design

- Data structure can vary from one record to another.
- No need to define tables before storing data.

2. Horizontal Scalability

- Capacity increases by adding more servers.
- Supports large-scale distributed systems.

3. High Availability

- Data is replicated across multiple nodes.
- System remains operational even if a server fails.

4. Distributed Architecture

- Data is stored on multiple machines.
- Improves reliability and performance.

5. Support for Multiple Data Models

- Key-Value Model
- Document Model
- Column-Family Model
- Graph Model

6. High Performance

- Optimized for fast data access and processing.
- Suitable for real-time applications.

7. Fault Tolerance

- Continues operation despite hardware failures.
- Uses replication and data redundancy.

Advantages of NoSQL Databases

1. Scalability

- Easily scales by adding more servers.
- Handles growing data volumes efficiently.

2. Flexibility

- Supports dynamic and changing data structures.
- No fixed schema restrictions.

3. High Speed

- Faster data insertion and retrieval.
- Suitable for real-time applications.

4. Cost-Effective

- Can run on inexpensive commodity hardware.
- Reduces infrastructure costs.

5. High Availability

- Replication ensures continuous service.
- Minimizes downtime.

6. Better Handling of Big Data

- Designed to manage massive datasets.
- Supports distributed data storage.

7. Supports Diverse Data Types

- Structured data
- Semi-structured data
- Unstructured data

Aggregate Data Models

An **Aggregate Data Model** is a data modeling approach in which related data items are grouped together and treated as a single unit called an **aggregate**. Instead of storing data in multiple related tables, aggregate-oriented databases store associated information together, making data retrieval faster and simpler. **Key Points**

- An aggregate is a collection of related data treated as one unit.
- Commonly used in NoSQL databases.
- Reduces the need for complex joins.
- Improves performance and scalability.
- Suitable for distributed database systems.

Here, all order-related information is stored together as a single aggregate.

Example

Customer Order Aggregate:

```
Order
├── Order ID: 1001
├── Customer Name: John
├── Products:
│   ├── Laptop
│   └── Mouse
└── Total Amount: $1200
```

Importance of Aggregate Data Models

Aggregate data models play a crucial role in NoSQL databases by simplifying data management and improving system performance.

1. Efficient Data Access

- Related data is stored together.
- Fewer database operations are required.

2. Improved Performance

- Eliminates expensive join operations.
- Faster query execution.

3. Better Scalability

- Aggregates can be distributed across multiple servers.

- Supports horizontal scaling.

4. Simplified Data Management

- Easier to maintain and update related information.
- Reduces data complexity.

5. Supports Distributed Systems

- Aggregates become natural units for partitioning and replication.

Benefits of Aggregate-Oriented Design

Aggregate-oriented design focuses on organizing data into aggregates to optimize storage and access.

1. Reduced Database Joins

- Related information is stored together.
- Minimizes join operations.

2. Faster Read Operations

- Entire aggregate can be retrieved in a single query.

- Improves response time.

3. Easier Data Distribution

- Aggregates can be partitioned across servers.
- Supports distributed architectures.

4. Better Consistency Management

- Updates occur within a single aggregate.
- Reduces synchronization issues.

5. Improved Scalability

- Aggregates can be independently scaled.

- Handles growing workloads efficiently.

6. Simplified Development

- Data structure closely matches application objects.
- Easier for developers to work with.

Aggregates

An **Aggregate** is a collection of related data elements that are treated as a single unit for storage, retrieval, and updates in a database. In NoSQL databases, aggregates are often stored as a single document, record, or key-value structure. **Key Points**

- Basic building block of aggregate-oriented databases.
- Groups related data together.
- Acts as a single unit for operations.
- Improves performance and scalability.

All customer-related information forms one aggregate.

Example

Customer Aggregate:

```
Customer
├── Customer ID
├── Name
├── Address
├── Phone Number
└── Orders
```

Aggregate Boundaries

Aggregate boundaries define which data belongs inside an aggregate and which data should remain outside it. **Purpose of Aggregate Boundaries**

- Control data organization.
- Improve performance.
- Maintain consistency.
- Simplify updates.

Guidelines for Defining Boundaries

1. **Keep Closely Related Data Together:** Frequently accessed data should be within the same aggregate.
2. **Limit Aggregate Size:** Very large aggregates can reduce performance.
3. **Maintain Transaction Consistency:** Data requiring atomic updates should belong to the same aggregate.
4. **Separate Independent Data:** Unrelated information should be stored in different aggregates.

Order and Customer Profile may be stored as separate aggregates.

Example

```
Order Aggregate
├── Order Details
├── Product Details
└── Payment Information

Customer Profile
├── Customer Name
├── Address
└── Contact Information
```

Uses of Aggregates

Aggregates are widely used in NoSQL database systems for efficient data handling.

1. **Data Storage:** Stores related data as a single unit.
2. **Data Retrieval:** Retrieves complete information with one query.

3. **Data Updates:** Updates multiple related fields together.

4. **Data Partitioning:** Aggregates can be distributed across servers.

5. **Replication:** Replicated as complete units in distributed systems.

Common Applications

- E-commerce orders
- Customer profiles
- Banking transactions
- Social media posts
- Inventory management systems

Advantages of Aggregates

1. **Faster Queries** – Related data is stored together, reducing retrieval time.
2. **Improved Scalability** – Aggregates can be easily distributed across multiple servers.
3. **Better Performance** – Fewer joins and database operations improve efficiency.
4. **Simplified Transactions** – All related updates can be performed within a single aggregate.
5. **Easier Maintenance** – Organized data structures make management and updates simpler.
6. **Efficient Replication** – Complete aggregates can be replicated easily across database nodes.

Key-Value Data Model

The **Key-Value Data Model** is a NoSQL database model that stores data as a collection of **key-value pairs**, where each key is unique and is used to access its corresponding value. **Key Points**

- Simplest NoSQL data model.
- Each key uniquely identifies a value.
- Values can be simple data or complex objects.
- Provides fast data retrieval.
- Suitable for distributed systems.

Here, **101, 102, and 103** are unique keys used to retrieve their respective values.

Example

Key	Value
101	John
102	Alice
103	David

Popular Key-Value Databases

- Redis
- Amazon DynamoDB
- Voldemor
- Riak
- Berkeley DB

Structure of Key-Value Databases

A key-value database consists of two basic components:

1. Key

- Unique identifier.
- Used to locate data quickly.
- Cannot be duplicated.

2. Value

- Actual data associated with the key.
- Can be text, numbers, images, documents, or objects.

Structure

Key → Value

Example

```
Student_ID → {  
    Name : "Rahul",  
    Age : 20,  
    Course : "BCA"  
}
```

Working Process

1. Data is stored as key-value pairs.
2. User provides a key.
3. Database searches for the key.
4. Corresponding value is returned.
1. **Simple Structure** – Stores data as key-value pairs.
2. **Fast Data Access** – Retrieves data directly using unique keys.
3. **High Scalability** – Easily expands across multiple servers.

Features

4. **Flexible Data Storage** – Supports different types of data as values.
5. **Schema-less Design** – Does not require a predefined database schema.
6. **High Availability** – Ensures continuous operation through replication and fault tolerance.
7. **Efficient Performance** – Delivers high-speed processing for large-scale and real-time applications.

Advantages

1. **High Speed** – Direct key lookup enables rapid data access.
2. **Easy Scalability** – Additional servers can be added easily.
3. **Simplicity** – Easy to design, implement, and maintain.
4. **Flexibility** – Supports various data formats without restrictions.

5. **Cost-Effective** – Runs efficiently on low-cost hardware.
6. **High Availability** – Replication reduces downtime and data loss.

Applications

1. **Caching Systems** – Store frequently accessed data for faster retrieval.
2. **Session Management** – Maintain user session information in web applications.
3. **User Profiles** – Store and retrieve user information quickly.
4. **Shopping Carts** – Manage temporary customer purchase data.
5. **Gaming Applications** – Store player profiles, scores, and game states.
6. **Real-Time Analytics** – Process and analyze high-speed data efficiently.

Document Data Model

The **Document Data Model** is a NoSQL data model that stores data in the form of documents, typically using JSON, BSON, or XML formats.

- Data is stored as self-contained documents.
- Each document contains related information together.
- Documents can have different structures.
- No fixed schema is required.
- Commonly used in NoSQL databases like MongoDB.

Document Structure

1. A document consists of field-value pairs.
2. Fields represent attributes of the data.
3. Values can be text, numbers, arrays, or nested documents.
4. Each document has a unique identifier.
5. Documents are usually stored in collections.

```
{  
  "StudentID": 101,  
  "Name": "Rahul",  
  "Age": 20,  
  "Course": "BCA"  
}
```

Features

1. **Schema-less Design** – Documents can have different structures.
2. **Flexible Storage** – Supports structured and semi-structured data.
3. **Nested Data Support** – Documents can contain sub-documents and arrays.
4. **High Scalability** – Easily distributed across multiple servers.
5. **Fast Retrieval** – Related data is stored within a single document.
6. **Self-Describing Data** – Documents include both data and structure.
7. **Easy Modification** – New fields can be added without redesigning the database.

Advantages

1. **Flexibility** – Supports changing data requirements without schema modifications.
2. **High Performance** – Reduces the need for complex joins.
3. **Scalability** – Supports horizontal scaling across servers.
4. **Easy Development** – Closely matches application objects and data structures.
5. **Efficient Storage** – Stores related data in a single document.
6. **Quick Access** – Retrieves complete records with fewer database operations.
7. **Cost-Effective** – Works efficiently on distributed commodity hardware.

Applications

1. **Content Management Systems** – Store articles, blogs, and media content.
2. **E-Commerce Platforms** – Manage product catalogs and customer information.
3. **User Profile Management** – Store user details and preferences.
4. **Mobile Applications** – Handle dynamic and rapidly changing data.
5. **Social Media Platforms** – Store posts, comments, and user interactions.
6. **Real-Time Analytics** – Process large volumes of semi-structured data.
7. **Catalog Management Systems** – Maintain product and inventory information.

Relationships in NoSQL Databases

A **relationship** in a NoSQL database represents the association between two or more data entities.

- Relationships connect related pieces of data.
- They show how one entity is linked to another.
- NoSQL handles relationships differently from relational databases.
- Relationships can be embedded or referenced.
- Proper relationship design improves data access efficiency.

A. Embedded Relationships: An **embedded relationship** stores related data inside the same document.

- Related data is stored within a parent document.
- Data can be retrieved in a single database query.
- Reduces the need for joins and lookups.
- Suitable for closely related data.
- Improves read performance.

```
{
  "CustomerID": 101,
  "Name": "Rahul",
  "Orders": [
    {"OrderID": 1, "Amount": 500},
    {"OrderID": 2, "Amount": 800}
  ]
}
```

Here, order details are embedded within the customer document.

B. Reference Relationships: A **reference relationship** stores related data in separate documents and links them using identifiers.

- Related data is stored in different documents.
- Documents are connected using reference IDs.
- Reduces data duplication.
- Suitable for large or frequently changing data.
- Requires additional queries to retrieve related data.

Here, CustomerID acts as a reference between the documents.

```
Customer
{
  "CustomerID": 101,
  "Name": "Rahul"
}

Order
{
  "OrderID": 1,
  "CustomerID": 101,
  "Amount": 500
}
```

Relationship Handling in NoSQL

1. NoSQL databases avoid complex joins used in relational databases.
2. Embedded relationships are used for tightly connected data.
3. Reference relationships are used for independent or large datasets.
4. Relationship handling depends on application requirements.
5. Aggregates help manage related data efficiently.
6. Denormalization is often used to improve performance.
7. Different NoSQL databases use different approaches to manage relationships.

Graph Databases

A Graph Database is a type of NoSQL database that stores data in the form of a graph. Instead of organizing data into tables and rows like relational databases, graph databases use nodes, edges, and properties to represent and store information. They are specifically designed to handle highly connected data and make relationship-based queries faster and more efficient.

Graph databases are widely used in applications where relationships between data elements are as important as the data itself, such as social networks, recommendation systems, and fraud detection systems.

Nodes

Nodes are the fundamental units of a graph database. They represent entities, objects, or records within the database. Each node contains information about a specific item.

For example:

- A person in a social network.
- A product in an e-commerce website.
- A city in a transportation network.

```
Node: Student
Name = Rahul
Roll No = 101
Department = CSE
```

Each node can store multiple attributes or properties that describe the entity it represents.

Edges

Edges, also known as relationships, connect two nodes and describe how they are related. Unlike relational databases where relationships are established through foreign keys, graph databases store relationships directly as edges.

Examples:

- Student "ENROLLED IN" Course

- Customer "PURCHASED" Product
- User "FOLLOWS" User

Since relationships are stored explicitly, graph databases can traverse connected data much faster than traditional databases.

Example: Rahul ----ENROLLED_IN----> Database Course

Properties

Properties are key-value pairs used to store additional information about nodes and edges. They provide details and characteristics of entities and relationships.

Examples of node properties:	Examples of edge properties:
<pre> Student ----- Name : Rahul Age : 21 Department : CSE </pre>	<pre> ENROLLED_IN ----- Semester : 5 Grade : A </pre>

Features

- Stores data as nodes, edges, and properties.
- Efficiently handles highly connected data.
- Supports flexible and dynamic schema design.
- Provides fast relationship traversal and querying.
- Eliminates the need for complex table joins.
- Offers high performance for network-based applications.
- Supports real-time analysis of relationships.

Advantages

- Faster execution of relationship-based queries.
- Better performance for connected data.
- Flexible structure allows easy modification.

- Simplifies representation of complex networks.
- Reduces query complexity by avoiding joins.
- Supports scalability for large datasets.
- Improves visualization of interconnected information.

Applications

- **Social Networks:** Models user relationships and interactions.
- **Recommendation Systems:** Provides personalized recommendations.
- **Fraud Detection:** Detects suspicious connections and activities.
- **Network Management:** Manages communication and computer networks.
- **Knowledge Graphs:** Stores interconnected knowledge.
- **Route Planning:** Finds optimal travel routes.

Schemeless Databases

A Schemeless Database is a type of database that does not require a predefined schema or fixed structure for storing data. Unlike traditional relational databases, where tables and columns must be defined before data insertion, schemeless databases allow records to have different structures within the same collection.

These databases are commonly associated with NoSQL systems and are designed to handle unstructured and semi-structured data efficiently. The flexible nature of schemeless databases makes them suitable for modern applications where data requirements change frequently.

Example:

Both documents can exist in the same collection even though they contain different fields.

```
Document 1:
{
  "Name": "Rahul",
  "Age": 21,
  "Department": "CSE"
}

Document 2:
{
  "Name": "Priya",
  "Department": "IT",
  "City": "Kolkata"
}
```

Characteristics

- **Flexible Schema:** No fixed structure is required.
- **Dynamic Data Model:** Records can have different fields.
- **Unstructured Data Support:** Stores diverse data formats.
- **Easy Scalability:** Supports distributed storage.
- **Rapid Development:** Simplifies application development.
- **High Availability:** Provides fault tolerance and reliability.

Challenges

- **Data Inconsistency:** Difficult to maintain uniformity.
- **Weak Validation:** Allows invalid data entries.
- **Complex Management:** Harder to organize data.
- **Query Issues:** Can affect query efficiency.
- **Data Redundancy:** May create duplicate data.
- **Governance Concerns:** Requires extra control measures.

Advantages

- **Flexibility:** Easily adapts to changes.
- **Faster Development:** Reduces design effort.
- **Diverse Data Handling:** Supports multiple data types.
- **Scalability:** Handles growing data volumes.
- **Reduced Maintenance:** Minimizes schema updates.
- **Agile Support:** Fits rapidly changing environments.

Applications

- **CMS:** Stores content and media files.
- **Social Media:** Manages user-generated content.
- **E-Commerce:** Stores diverse product data.
- **Big Data:** Handles massive datasets.
- **IoT:** Stores sensor-generated data.
- **Web and Mobile Apps:** Supports dynamic applications.
- **Real-Time Analytics:** Processes live data streams.

Materialized Views

A Materialized View is a database object that stores the result of a query physically in the database. Unlike a normal view, which executes the underlying query each time it is accessed, a materialized view stores the query results and retrieves them directly when requested.

Materialized views are primarily used to improve query performance, especially when dealing with large datasets, complex joins, aggregations, and frequently executed queries.

For example, if a company frequently generates a report showing total sales by region, the result can be stored in a materialized view instead of recalculating it every time the report is requested.

Working of Materialized Views

A materialized view is created based on a query involving one or more tables. When the view is created, the query is executed and its result is stored physically on disk.

Whenever users access the materialized view, the database retrieves the stored data directly rather than executing the original query again. This significantly reduces query execution time.

However, when the data in the underlying tables changes, the materialized view may become outdated. Therefore, it must be refreshed periodically to synchronize its contents with the source tables.

Steps in Working

1. Data is stored in one or more base tables.
2. A query is executed to create a materialized view.
3. The query result is physically stored in the database.
4. Users access the stored results directly.
5. The materialized view is refreshed when underlying data changes.

Example: Instead of calculating totals repeatedly, the database stores the result in the materialized view.

Sales Table	

Region	Sales
East	50000
West	40000
North	35000
Materialized View	

Region-wise Total Sales	

Benefits

- **Improved Performance:** Speeds up query execution.
- **Reduced Cost:** Lowers processing overhead.
- **Faster Reporting:** Generates reports quickly.
- **Large Database Support:** Improves system efficiency.
- **Data Warehousing:** Enhances analytical processing.
- **Reduced Traffic:** Minimizes data transfer.

Applications

- **Data Warehousing:** Stores summarized data.
- **Business Intelligence:** Supports dashboards and reports.
- **Decision Support:** Aids business decisions.
- **Financial Reporting:** Generates financial summaries.
- **E-Commerce Analytics:** Analyzes sales and customers.
- **Performance Optimization:** Speeds up frequent queries.

Module 2.2: Distributed Data Models

Contents

NoSQL Databases	Error! Bookmark not defined.
Need for NoSQL Databases	Error! Bookmark not defined.
Characteristics of NoSQL Databases	Error! Bookmark not defined.
Advantages of NoSQL Databases	Error! Bookmark not defined.
Aggregate Data Models	Error! Bookmark not defined.
Importance of Aggregate Data Models	Error! Bookmark not defined.
Benefits of Aggregate-Oriented Design	Error! Bookmark not defined.
Aggregates	Error! Bookmark not defined.
Aggregate Boundaries	Error! Bookmark not defined.
Uses of Aggregates	Error! Bookmark not defined.
Advantages of Aggregates	Error! Bookmark not defined.
Key-Value Data Model	Error! Bookmark not defined.
Document Data Model	Error! Bookmark not defined.
Relationships in NoSQL Databases	Error! Bookmark not defined.
Graph Databases	Error! Bookmark not defined.
Nodes	Error! Bookmark not defined.
Edges	Error! Bookmark not defined.
Properties	Error! Bookmark not defined.
Schemeless Databases	Error! Bookmark not defined.
Materialized Views	Error! Bookmark not defined.
Working of Materialized Views	Error! Bookmark not defined.

Distributed Data Models

A Distributed Data Model is a database model in which data is stored across multiple physical locations, servers, or sites connected through a network. Although the data is distributed, it appears to users as a single unified database. Distributed data models improve performance, availability, reliability, and scalability by sharing data among different locations.

Need for Data Distribution

Data distribution is necessary in modern database systems for several reasons:

- **Improved Performance:** Data can be accessed from the nearest location, reducing response time.
- **Scalability:** Additional servers can be added to handle increasing data and user demands.
- **High Availability:** Data remains accessible even if one site or server fails.
- **Reliability:** Multiple copies of data reduce the risk of data loss.
- **Load Balancing:** Workload can be distributed across multiple servers.
- **Geographical Distribution:** Supports organizations operating in multiple locations.
- **Reduced Network Traffic:** Local access to data minimizes network congestion.

Types of Distribution Models

A. Partitioned (Fragmented) Distribution

Model: In this model, the database is divided into smaller fragments, and each fragment is stored at a different site. Each site manages only its assigned portion of the data.

Advantages:

- Reduces storage requirements at individual sites.
- Improves query performance for local users.
- Supports efficient data management.

Example: Customer records are divided among branches based on geographical regions.

B. Replicated Distribution Model: In this model, copies of the same data are stored at multiple sites. Every site contains identical or partially identical data. **Advantages:**

- Increases data availability.
- Improves fault tolerance.

- Provides faster access to frequently used data.

Example: A bank stores copies of customer account information at multiple branch servers.

C. Hybrid Distribution Model: The hybrid model combines both fragmentation and replication. Some data is partitioned across sites, while important data is replicated at multiple locations. **Advantages:**

- Provides both high performance and reliability.
- Balances storage efficiency and availability.
- Supports large-scale distributed systems.

Example: Regional sales data is fragmented by location, while product information is replicated across all branches.

Sharding

Sharding is a database scaling technique in which a large database is divided into smaller, independent parts called **shards**. Each shard contains a portion of the total data and is stored on a separate server. Together, all shards form a single logical database.

Sharding is commonly used in large-scale distributed databases to improve performance, scalability, and data management.

Sharding Process: The sharding process involves dividing data and distributing it across multiple database servers.

Steps in Sharding:

1. Select a **shard key** that determines how data will be distributed.
2. Divide the database into multiple shards based on the shard key.
3. Store each shard on a separate server or node.
4. Route incoming queries to the appropriate shard.
5. Combine results from different shards when necessary.

```
Customer Database
Shard 1 → Customers A-F
Shard 2 → Customers G-M
Shard 3 → Customers N-Z
```

Types of Sharding

i) Range-Based Sharding: In range-based sharding, data is divided according to a specified range of values in the shard key.

Advantages:

- Simple to implement.
- Efficient for range queries.

Disadvantages:

- Uneven data distribution may occur.
- Some shards may become overloaded.

```
Shard 1 → IDs 1-1000
Shard 2 → IDs 1001-2000
Shard 3 → IDs 2001-3000
```

ii) Hash-Based Sharding: In hash-based sharding, a hash function is applied to the shard key to determine the target shard.

Example: $\text{Shard Number} = \text{Hash}(\text{Customer_ID}) \% \text{Number of Shards}$

Advantages:

- Provides uniform data distribution.

- Prevents hotspot formation.

Disadvantages:

- Range queries become difficult.
- Resharding can be complex.

iii) Directory-Based Sharding: In directory-based sharding, a lookup table or directory maintains information about the location of each data item. The directory is consulted before accessing data.

Advantages:

- Flexible data placement.
- Easy to modify shard assignments.

- Requires maintenance of the directory.
- Directory failures can affect performance.

Disadvantages:

```
Customer A → Shard 1
Customer B → Shard 3
Customer C → Shard 2
```

Advantages

- **Scalability:** Supports growth by adding more servers.
- **Improved Performance:** Reduces workload on individual servers.
- **High Availability:** Failure of one shard does not affect the entire database.
- **Load Balancing:** Distributes data and traffic across multiple nodes.
- **Faster Queries:** Smaller datasets improve query execution speed.
- **Cost Efficiency:** Uses multiple low-cost servers instead of a single powerful server.

Challenges

- **Complex Management:** Shard administration can be difficult.
- **Data Rebalancing:** Redistribution is required when adding new shards.
- **Cross-Shard Queries:** Queries spanning multiple shards are more complex.
- **Uneven Distribution:** Some shards may store more data than others.
- **Backup Complexity:** Managing backups across shards can be challenging.
- **Increased Development Effort:** Applications must be designed to support sharding.

Replication

Replication is the process of creating and maintaining multiple copies of the same data across different database servers or nodes. The copies, known as replicas, are kept synchronized so that changes made to one copy are reflected in the others.

Replication is used in distributed database systems to improve data availability, reliability, fault tolerance, and performance. By storing data at multiple locations, replication ensures that the system can continue operating even if one server fails.

b) Need for Replication

Replication is important for several reasons:

- Improves data availability.
- Provides backup copies of data.
- Increases fault tolerance.
- Reduces the risk of data loss.
- Improves read performance.
- Supports disaster recovery.
- Enables load balancing.
- Ensures business continuity.

- Allows data access from multiple locations.
- Enhances system reliability.

c) Working of Replication

The replication process generally follows these steps:

1. Data is stored in a primary database.
2. Changes are made to the primary database.
3. The changes are recorded in transaction logs.
4. Replication mechanisms capture these changes.
5. Updates are transmitted to replica databases.
6. Replica databases apply the changes.
7. All copies remain synchronized.

Master-Slave Replication

Master-Slave Replication is a database replication technique in which one server, called the **Master**, manages all data modifications, while one or more **Slave** servers maintain copies of the master's data. The slaves receive updates from the master and keep their databases synchronized. This model is widely used to improve read performance, provide backup copies of data, and ensure higher availability in distributed database systems.

b) Architecture

The architecture consists of a central Master node and multiple Slave nodes.

Master Server

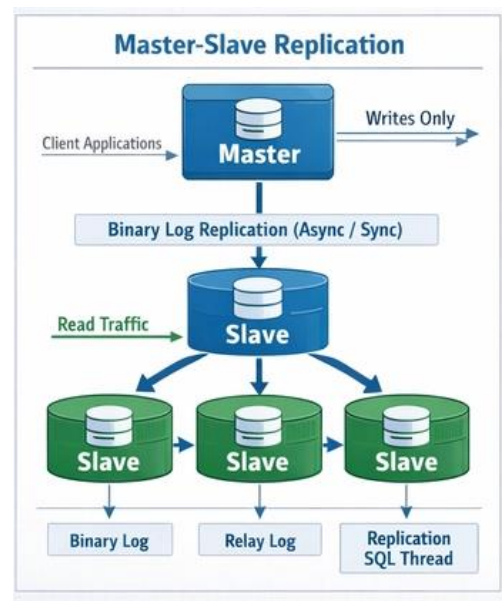
- Stores the primary copy of data.
- Handles INSERT, UPDATE, and DELETE operations.
- Records changes in transaction logs.
- Sends updates to Slave servers.

Slave Servers

- Maintain copies of the master's database.
- Receive updates from the Master.
- Primarily handle read requests.
- Provide backup and redundancy.

c) Working

- A client sends a write request to the Master server.
- The Master updates its database.
- The Master records the changes in a transaction log.
- Replication software transfers these changes to Slave servers.



- Each Slave applies the changes to its local database.
- Read operations can be performed from any Slave server.
- The process continues to keep all replicas synchronized.

d) Advantages

- Improves read performance by distributing queries among slaves.
- Reduces workload on the Master server.
- Provides backup copies of data.
- Enhances system availability.
- Supports disaster recovery.
- Easy to understand and implement.
- Allows load balancing of read operations.
- Improves response time for users.
- Suitable for large web applications.

- Helps maintain business continuity.

e) Limitations

- Master becomes a single point of failure.
- Write operations depend entirely on the Master.
- Replication delays may occur.
- Slaves may temporarily contain outdated data.
- Additional storage is required for replicas.
- Failover management can be complex.
- Not suitable for write-intensive applications.

- Network failures can affect synchronization.
- Recovery after Master failure may take time.
- Scalability of write operations is limited.

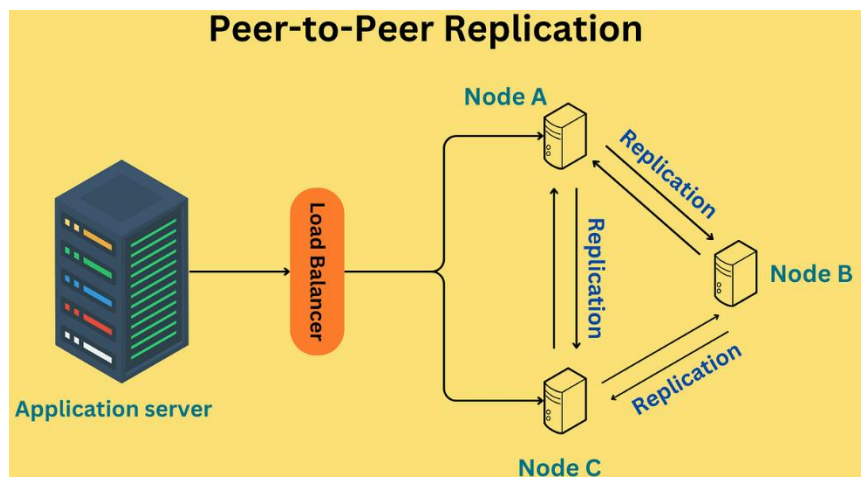
f) Applications of Master-Slave Replication

- E-commerce websites.
- Banking systems.
- Online reservation systems.
- Content management systems.
- Social networking platforms.
- Data warehousing environments.
- Large-scale web applications.
- Reporting and analytics systems.

Peer-to-Peer Replication

Peer-to-Peer Replication is a replication model in which all database nodes have equal status. Each node can perform both read and write operations and can share updates with other nodes.

Unlike Master-Slave Replication, there is no central controlling server. Every node acts as both a publisher and subscriber of data.



b) Architecture

The architecture consists of multiple interconnected database nodes.

- No dedicated Master server exists.
- Every node contains a copy of the database.
- Each node can process read and write requests.

- Data updates are exchanged among all participating nodes.
- Nodes communicate directly with one another.

c) Working

1. A user updates data on any node.
2. The node records the modification.

3. The update is transmitted to other nodes.
4. All nodes apply the received changes.
5. Data remains synchronized across the network.
6. If two nodes modify the same data simultaneously, conflict resolution mechanisms are applied.
7. Each node eventually reaches a consistent state.

d) Advantages

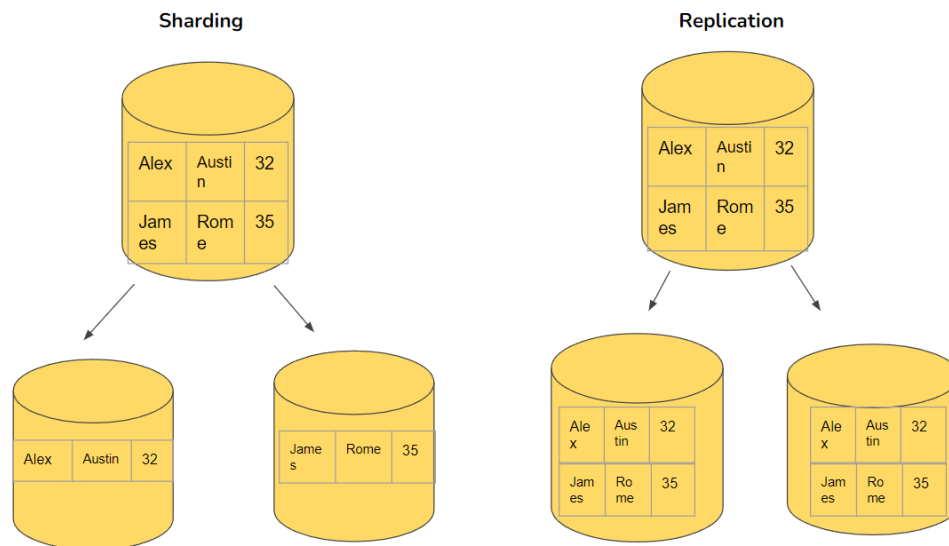
- Eliminates a single point of failure.
- Provides high system availability.
- Supports both read and write operations at every node.
- Improves fault tolerance.
- Enables geographical distribution of databases.
- Reduces dependency on a central server.
- Allows local access to data.
- Supports load sharing among nodes.

- Increases system reliability.
- Suitable for distributed enterprise environments.

e) Limitations

- Conflict detection and resolution can be difficult.
- Synchronization mechanisms are complex.
- Network traffic increases significantly.
- Maintaining data consistency is challenging.
- Administration becomes more complicated.
- Performance may decrease as the number of nodes grows.
- Troubleshooting replication issues can be difficult.
- Requires careful planning and monitoring.
- Higher implementation cost.
- Not ideal for applications requiring strict real-time consistency.

Basis	Replication	Sharding
Definition	Creates multiple copies of the same data.	Divides data into smaller parts called shards.
Purpose	Improves availability and fault tolerance.	Improves scalability and performance.
Data Storage	Same data exists on multiple servers.	Different data exists on different servers.
Data Distribution	Data is duplicated across nodes.	Data is partitioned across nodes.
Read Performance	Improved due to multiple replicas.	Improved as queries access smaller datasets.
Write Performance	Usually limited by synchronization.	Improved by distributing write operations.
Storage Requirement	High, due to duplicate copies.	Lower, as data is split among servers.
Fault Tolerance	High, because replicas act as backups.	Lower unless combined with replication.
Scalability	Mainly supports read scalability.	Supports both data and workload scalability.
Data Availability	High availability during failures.	Depends on shard availability.
Consistency Issues	Replication lag may occur.	Cross-shard consistency can be challenging.
Network Overhead	High due to synchronization traffic.	Lower compared to replication.
Implementation Complexity	Relatively simpler.	More complex due to partition management.
Example	Master-Slave Replication.	Range-Based or Hash-Based Sharding.

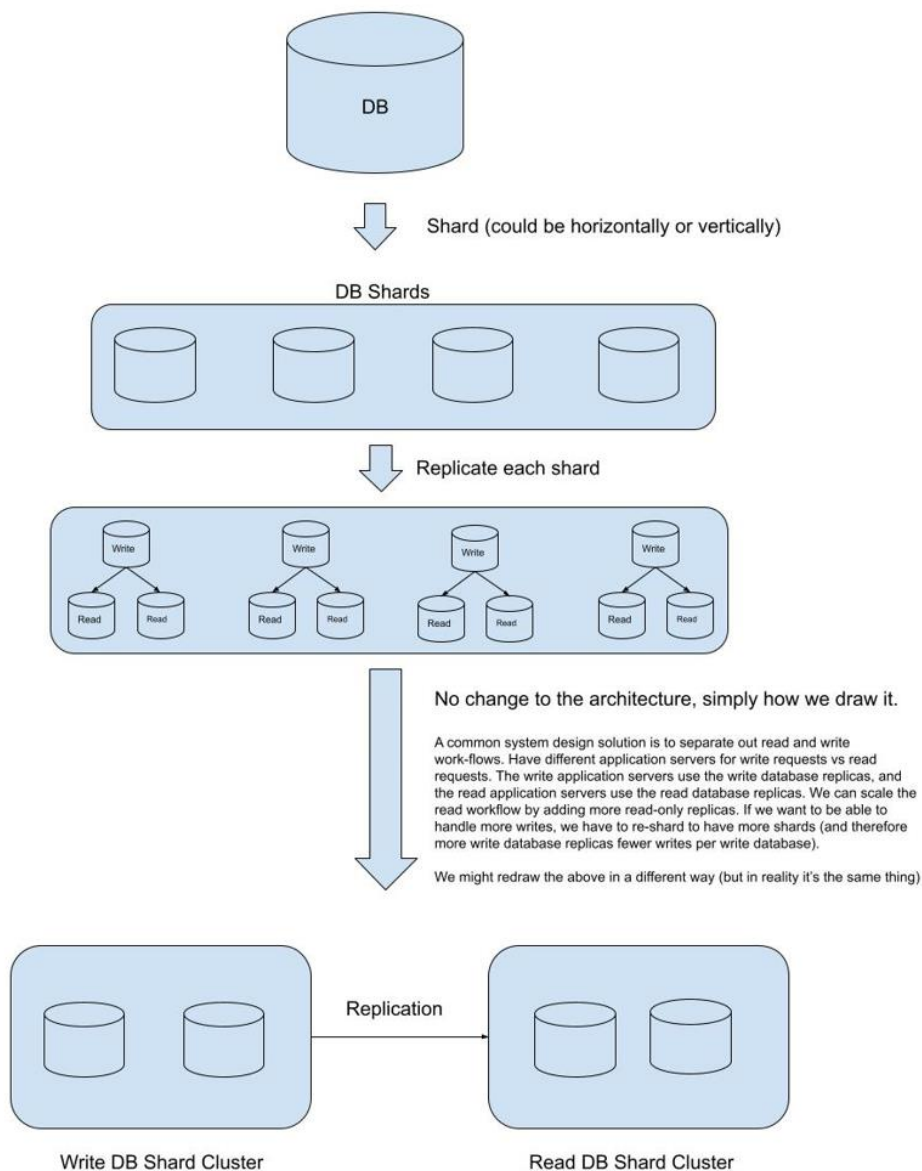


Combining Sharding and Replication

Sharding and Replication are often used together in distributed database systems to achieve both scalability and high availability. **Sharding** divides a large database into smaller partitions called **shards**, while **Replication** creates multiple copies of each shard on different servers.

In this approach:

- Data is first divided into multiple shards.
- Each shard is stored on a separate server.
- Copies of each shard are maintained on replica servers.
- One replica may act as the primary node, while others act as secondary nodes.
- Queries are distributed across shards and replicas.



a) Working

1. Data is divided into multiple shards.
2. Each shard is assigned to a server.
3. Copies of every shard are created on replica servers.
4. Write operations are performed on the primary replica.
5. Changes are synchronized to secondary replicas.
6. Read requests may be served by replicas.
7. If one server fails, a replica can take over.

This combination provides both horizontal scaling and fault tolerance.

b) Benefits of the combination

- **Scalability:** Supports growth by adding new shards.
- **High Availability:** Replicas keep data accessible during failures.
- **Better Performance:** Queries run faster on smaller datasets.
- **Load Balancing:** Workload is distributed across servers.
- **Fault Tolerance:** System continues operating despite node failures.

- **Data Redundancy:** Multiple copies reduce data loss risk.
- **Faster Reads:** Replicas handle read requests efficiently.
- **Geographic Distribution:** Data can be stored closer to users.
- **Improved Reliability:** Multiple nodes increase system dependability.
- **Supports Big Data:** Handles massive volumes of data effectively.

- **Higher Storage Cost:** Replicas require additional storage space.
- **Network Overhead:** Synchronization generates extra network traffic.
- **Failover Complexity:** Recovery procedures can be difficult to manage.
- **Monitoring Requirements:** Multiple servers require continuous monitoring.
- **Maintenance Effort:** System administration becomes more demanding.

c) Challenges

- **Complex Management:** Combining sharding and replication increases complexity.
- **Cross-Shard Queries:** Queries across shards are difficult to execute.
- **Data Rebalancing:** Shards must be redistributed when scaling.
- **Replication Lag:** Replicas may not reflect updates immediately.
- **Consistency Issues:** Maintaining identical data across replicas is challenging.

d) Applications of Sharding with Replication

- Large-scale social media platforms.
- E-commerce websites.
- Cloud databases.
- Banking and financial systems.
- Online gaming platforms.
- Big data analytics systems.
- Streaming services.
- Global web applications.

Consistency

Consistency is the property of a distributed database that ensures all users see the same and most recent data across all nodes. After a data update, every replica should eventually reflect the same value. Consistency ensures that data remains accurate, valid, and synchronized throughout the system.

b) Importance of Consistency

- Ensures data accuracy across all nodes.
- Prevents conflicting data values.
- Maintains data integrity.
- Improves reliability of transactions.
- Provides a uniform view of data.
- Reduces chances of incorrect decisions.
- Supports application correctness.
- Essential for financial and banking systems.
- Helps maintain user trust.
- Ensures proper synchronization of replicas.

c) Consistency Models

Consistency models define how and when updates become visible to users in a distributed database.

i) Strong Consistency

Strong Consistency guarantees that every read operation returns the most recent write. Once data is updated, all users immediately see the updated value regardless of the node they access.

Characteristics

- Always returns the latest data.
- Ensures a single consistent view.
- No stale data is observed.
- All nodes remain synchronized.

Advantages

- High data accuracy.
- Simplifies application development.
- Suitable for critical applications.
- Prevents data conflicts.

Limitations

- Higher response time.
- Increased communication overhead.
- Reduced system availability during failures.

Applications

- Banking systems.
- Online payment systems.
- Stock trading platforms.
- Reservation systems.

ii) Weak Consistency

Weak Consistency does not guarantee that all users immediately see the latest update. Data updates may take time to propagate across nodes.

Characteristics

- Updates are not instantly visible.
- Different nodes may contain different values temporarily.
- Consistency is not guaranteed after every write.

Advantages

- Faster response time.
- Lower communication overhead.
- Better system performance.

Limitations

- Users may read outdated data.
- Temporary inconsistencies may occur.
- Not suitable for critical transactions.

Applications

- Caching systems.
- Multimedia streaming.
- Social media feeds.
- Online gaming systems.

iii) Eventual Consistency

Eventual Consistency is a consistency model in which all replicas eventually become consistent if no new updates occur. Temporary inconsistencies are allowed, but the system guarantees convergence over time.

Characteristics

- Updates propagate gradually.
- Temporary inconsistencies are allowed.
- All replicas eventually reach the same state.
- Common in large-scale distributed systems.

Advantages

- High availability.
- Better scalability.
- Improved performance.

- Supports geographically distributed databases.

Limitations

- Users may temporarily see stale data.
- Conflict resolution may be required.
- Consistency is not immediate.

Applications

- NoSQL databases.
- Cloud storage systems.
- E-commerce platforms.
- Distributed web applications.
- Social networking platforms.

Comparison of Consistency Models

Feature	Strong Consistency	Weak Consistency	Eventual Consistency
Latest Data Guarantee	Always	No	Eventually
Data Accuracy	Very High	Low	Moderate
Response Time	Slower	Faster	Fast
Availability	Lower	Higher	High
Scalability	Limited	High	Very High
Temporary Inconsistency	Not Allowed	Allowed	Allowed
Suitable For	Banking, Payments	Caching, Gaming	NoSQL, Cloud Systems

Relaxing Consistency

Relaxing Consistency is an approach in distributed database systems where strict consistency requirements are reduced to improve system performance, availability, and scalability. Instead of ensuring that all nodes immediately reflect every update, temporary inconsistencies are allowed, with the expectation that data will become consistent later.

This approach is widely used in large-scale distributed systems where immediate consistency is less important than system responsiveness and availability.

b) Need for Relaxing Consistency

- Improves system availability.
- Reduces response time.
- Enhances scalability.
- Supports geographically distributed systems.
- Minimizes communication overhead.
- Handles network delays efficiently.
- Improves user experience.
- Supports large numbers of users.
- Reduces synchronization costs.
- Enables continuous system operation during failures.
- Improves system performance.
- Supports horizontal scalability.
- Reduces network overhead.
- Allows faster data access.
- Handles large workloads efficiently.
- Supports distributed environments.
- Improves fault tolerance.

c) How Relaxing Consistency Works

1. Data is updated on one node.
2. The update is accepted immediately.
3. Other nodes receive the update later.
4. Temporary differences may exist among replicas.
5. Synchronization occurs in the background.
6. All replicas eventually become consistent.

d) Advantages

- Provides high availability.

e) Challenges

- Users may read stale data.
- Temporary inconsistencies can occur.
- Conflict resolution may be required.
- Data synchronization becomes complex.
- Application logic may be more difficult.
- Consistency guarantees are reduced.
- Debugging can be challenging.
- Not suitable for critical transactions.

f) Applications

- Social networking platforms.
- Cloud storage systems.
- NoSQL databases.
- E-commerce websites.
- Content delivery networks (CDNs).
- Online gaming platforms.

- Distributed web applications.
- Large-scale internet services.

Example

Suppose a user updates their profile picture on a social media platform.

- The update is saved on one server immediately.

- Some users may still see the old picture for a short time.
- After synchronization, all servers display the new picture.
- The system achieves consistency eventually rather than instantly.

Version Stamps

A Version Stamp is a unique identifier or version number associated with a data item to track changes made to it over time. Whenever the data is modified, its version stamp is updated.

Version stamps help distributed databases determine whether data has changed and identify the most recent version of a record.

b) Purpose

- Tracks changes made to data.
- Identifies the latest version of a record.
- Detects data conflicts.
- Supports synchronization between replicas.
- Maintains data consistency.
- Prevents overwriting of recent updates.
- Assists in concurrency control.
- Improves data integrity.

c) Working

1. A record is assigned an initial version stamp.
2. When the record is updated, the version stamp changes.
3. Replicas compare version stamps during synchronization.
4. The record with the newer version stamp is identified.
5. Conflicts can be detected using version differences.
6. Replicas update themselves with the latest version.
7. Consistency is maintained across the system.

Example

Record	Data	Version Stamp
Customer 101	Kolkata	V1
Customer 101	Mumbai	V2

Here, **V2** indicates that the record has been updated after **V1** and is therefore the latest version.

d) Advantages

- Easily identifies updated records.
- Simplifies synchronization.
- Helps detect update conflicts.
- Supports distributed databases.
- Improves consistency management.
- Prevents data overwrite issues.
- Enhances concurrency control.
- Reduces synchronization errors.
- Maintains data integrity.
- Improves reliability of replicated systems.

Module 3.1: Hadoop Architecture and HDFS

Contents

Data Formats	1
Analysing Data with Hadoop	2
Scaling Out	3
Hadoop Streaming	4
Hadoop Pipes	4
Hadoop Distributed File System (HDFS)	5
HDFS Java Interface.....	6
HDFS Data Flow	7

Data Formats

Data format refers to the way data is organized, stored, and represented in a computer system. The main types of data formats are:

- 1. Structured Data Formats:** Structured data is organized in a fixed format with predefined rows and columns.
 - Follows a predefined schema.
 - Stored in tables with rows and columns.
 - Easy to search and analyze.
 - Commonly used in relational databases.

Examples

- SQL Database Tables
- Excel Spreadsheets
- CSV Files

- 2. Semi-Structured Data Formats:** Semi-structured data does not follow a strict table structure but contains tags or metadata for organization.
 - Flexible schema.
 - Self-describing data.
 - Easier to modify than structured data.
 - Suitable for hierarchical information.

Examples

- XML
- JSON
- HTML

- 3. Unstructured Data Formats:** Unstructured data has no predefined format or organization.
 - Does not follow a fixed schema.
 - Difficult to process using traditional databases.
 - Large volume of data is generated daily.
 - Requires specialized tools for analysis.

Examples

- Images
- Audio Files
- Videos
- Emails
- Social Media Posts

Importance of Data Formats in Big Data

- Helps organize and store data efficiently.
- Improves data processing speed.
- Enables easy data integration from multiple sources.
- Supports analytics and decision-making.
- Reduces storage and management complexity.
- Helps select appropriate big data tools and technologies.

Analysing Data with Hadoop

Need for Data Analysis: Data analysis is the process of examining large amounts of data to discover useful information, patterns, and trends.

- Helps organizations make informed decisions.
- Identifies hidden patterns and relationships.
- Improves business performance and efficiency.
- Supports forecasting and prediction.
- Enables better customer understanding.
- Helps detect anomalies and fraud.

Hadoop for Data Processing

Hadoop is an open-source framework designed for storing and processing large datasets across clusters of computers.

Features of Hadoop for Data Processing

- Processes massive volumes of data efficiently.
- Uses distributed storage through HDFS.
- Employs MapReduce for parallel computation.
- Provides fault tolerance through replication.
- Supports scalability by adding more nodes.

Working of Hadoop Data Processing

1. Data is stored in HDFS.
2. Data is divided into blocks.
3. Blocks are distributed across DataNodes.
4. MapReduce processes the data in parallel.
5. Results are combined and returned to the user.

Batch Processing

Batch processing is a method of processing large volumes of data together as a single batch instead of processing data individually.

Characteristics

- Processes data in groups.
- Suitable for large datasets.
- Does not require immediate results.
- Efficient resource utilization.

- Bank statement generation
- Log file analysis

Advantages

- High throughput.
- Cost-effective.
- Suitable for repetitive tasks.

Examples

- Payroll processing

Parallel Processing

Parallel processing is the technique of executing multiple tasks simultaneously using multiple processors or machines.

- Divides work into smaller tasks.
- Executes tasks at the same time.
- Reduces processing time.
- Improves system performance.
- Data is distributed across multiple nodes.
- Each node processes its portion independently.
- Results are combined to produce the final output.

Hadoop and Parallel Processing

Advantages

- Faster execution.
- Better resource utilization.
- Supports large-scale data analysis.

Advantages of Hadoop in Data Analysis

- Handles very large datasets efficiently.
- Provides distributed storage and processing.
- Supports fault tolerance through data replication.
- Scales easily by adding more nodes.
- Processes data in parallel for faster results.
- Cost-effective because it uses commodity hardware.
- Supports structured, semi-structured, and unstructured data.
- Open-source and widely adopted in industry.

Scaling Out

Scaling Out is the process of increasing system capacity by adding more machines (nodes) to a distributed system instead of upgrading existing hardware. **Key Points**

- Also known as Horizontal Scaling.
- Commonly used in Hadoop clusters.
- Distributes workload among multiple nodes.
- Improves performance and storage capacity.

1) Horizontal Scaling: refers to adding additional servers or nodes to a system to handle increasing data and processing requirements. **Example:** If a Hadoop cluster with 5 nodes becomes overloaded, 5 more nodes can be added to increase processing power and storage.

- New nodes can be added easily.
- Workload is distributed across nodes.
- Provides better fault tolerance.
- Supports large-scale applications.

2) Vertical Scaling: refers to increasing the resources (CPU, RAM, storage, etc.) of an

existing server to handle increasing data and processing requirements. **Example:** If a server has 8 GB RAM and 4 CPU cores, it can be upgraded to 32 GB RAM and 16 CPU cores to improve performance.

- Hardware resources of a single machine are upgraded.
- Processing power and memory are increased.
- Easy to implement without adding new servers.
- Limited by the maximum capacity of the machine.

Scale Up (Vertical Scaling)	Scale Out (Horizontal Scaling)
Increases resources of an existing machine.	Adds more machines to the system.
Limited by hardware capacity.	Easily expandable by adding nodes.
More expensive hardware required.	Uses commodity hardware.
Single point of failure may exist.	Better fault tolerance.
Suitable for small systems.	Suitable for Big Data environments.

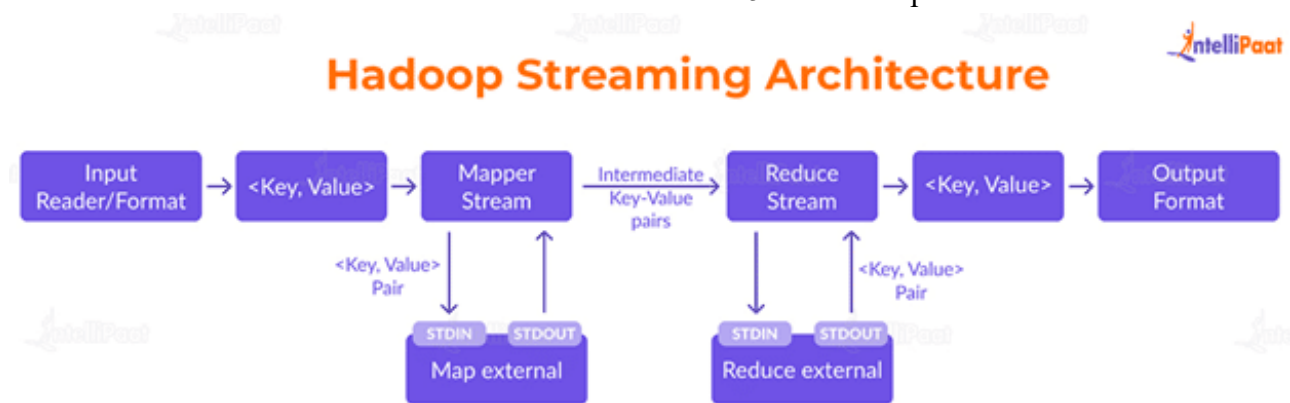
Benefits of Scaling Out

- Increases storage and processing capacity.
- Supports handling of massive datasets.
- Improves fault tolerance and reliability.
- Reduces system downtime.
- Cost-effective using commodity hardware.
- Enables parallel data processing.
- Provides flexible and easy expansion.

Hadoop Streaming

Hadoop Streaming is a utility that allows users to create MapReduce programs using any programming language that can read from standard input and write to standard output. **Examples of Languages:** Python, Ruby, Perl, Shell Scripts, PHP. **Steps**

1. Input data is provided to the Mapper.
2. Mapper processes the data using a chosen programming language.
3. Hadoop sorts and groups intermediate results.
4. Reducer processes grouped data.
5. Final output is stored in HDFS.



Features

- Supports multiple programming languages.
- Works with standard input and output.
- Easy integration with existing scripts.
- Compatible with Hadoop MapReduce.
- Suitable for rapid application development.

Advantages

- No need to write Java code.
- Supports language flexibility.

- Easy to learn and implement.
- Reuses existing programs and scripts.
- Faster development of MapReduce applications.

Applications

- Log file analysis.
- Text processing.
- Data transformation.
- Data filtering.
- Web data analysis.

Hadoop Pipes

Hadoop Pipes is a framework that allows developers to write Hadoop MapReduce applications in C++ instead of Java. **Working**

1. Input data is stored in HDFS.
2. Hadoop launches C++ Mapper programs.
3. Intermediate data is shuffled and sorted.

4. C++ Reducer programs process grouped data.
5. Output is stored in HDFS.

Features

- Supports MapReduce development in C++.
- Provides communication between Hadoop and C++ programs.
- Suitable for high-performance applications.
- Uses the Hadoop framework for execution.
- Integrates with HDFS and MapReduce.

Advantages

- Better performance for compute-intensive tasks.

- Allows reuse of existing C++ code.
- Suitable for developers familiar with C++.
- Provides efficient memory management.
- Supports large-scale distributed processing.

Applications

- Scientific computing.
- Data mining.
- Image processing.
- High-performance analytics.
- Large-scale computation tasks.

Hadoop Distributed File System (HDFS)

HDFS is a distributed file system designed to store very large files across multiple machines while providing high throughput and fault tolerance. **Design Components**

1) NameNode

- Master node of HDFS.
- Maintains metadata about files and directories.
- Keeps track of block locations.
- Handles client requests.
- Does not store actual file data.

2) DataNode

- Worker node of HDFS.
- Stores actual data blocks.
- Performs read and write operations.
- Sends heartbeat messages to the NameNode.
- Replicates blocks as instructed by the NameNode.

3) Blocks

- Files are divided into fixed-size blocks.

- Default block size is typically 128 MB or 256 MB.
- Blocks are distributed across DataNodes.
- Large files can be stored efficiently.

4) Replication

- Multiple copies of each block are stored.
- Default replication factor is 3.
- Ensures fault tolerance.
- Prevents data loss due to node failures.

5) Rack Awareness

- Hadoop places replicas across different racks.
- Improves fault tolerance.
- Reduces network congestion.
- Ensures data availability during rack failures.

Objectives of HDFS

- Store huge volumes of data efficiently.
- Provide fault tolerance through replication.

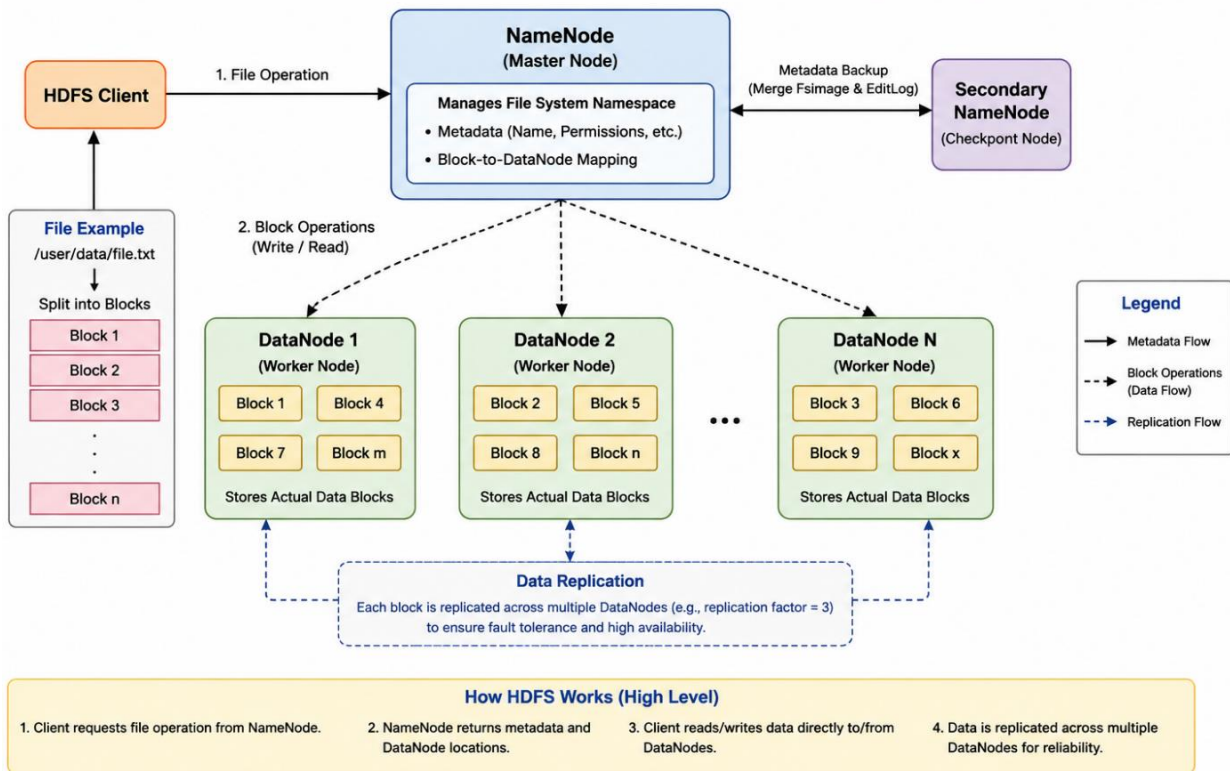
- Support distributed storage.
- Deliver high data throughput.
- Enable scalability by adding nodes.
- Ensure reliable data access.

- High availability and reliability.
- High-throughput data access.
- Automatic recovery from failures.
- Scalability through cluster expansion.
- Cost-effective storage using commodity hardware.

Key Features

- Distributed storage architecture.
- Large file support.
- Fault tolerance through replication.

HADOOP DISTRIBUTED FILE SYSTEM (HDFS)



HDFS Java Interface

The HDFS Java Interface provides APIs that allow Java applications to interact with HDFS for storing, retrieving, and managing files. **Uses**

- Access files in HDFS.
- Create and delete files.
- Read and write data.
- Manage directories.

File Operations

Common file operations supported by the HDFS Java API:

- Create files.
- Open files.
- Read files.
- Write files.
- Delete files.
- Rename files.
- Copy files.
- Create directories.

Example Operations

create()
open()

delete()
rename()

mkdirs()

Read/Write Operations

Read Operation

1. Client requests file from NameNode.
2. NameNode returns block locations.
3. Client reads blocks from DataNodes.
4. Data is returned to the application.

3. Client sends data to DataNodes.
4. Data is replicated.
5. Metadata is updated by NameNode.

Advantages

- Efficient data access.
- Supports large files.
- Automatic fault tolerance.
- Distributed storage management.

Write Operation

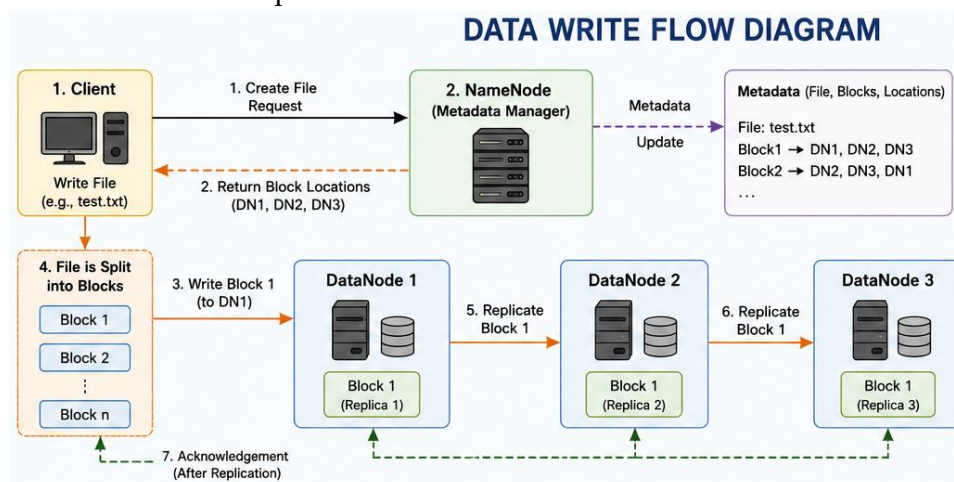
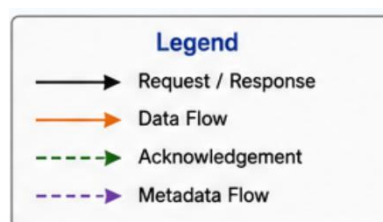
1. Client requests file creation.
2. NameNode allocates blocks.

HDFS Data Flow

Data Write Flow

When a client writes a file to HDFS, the following steps occur:

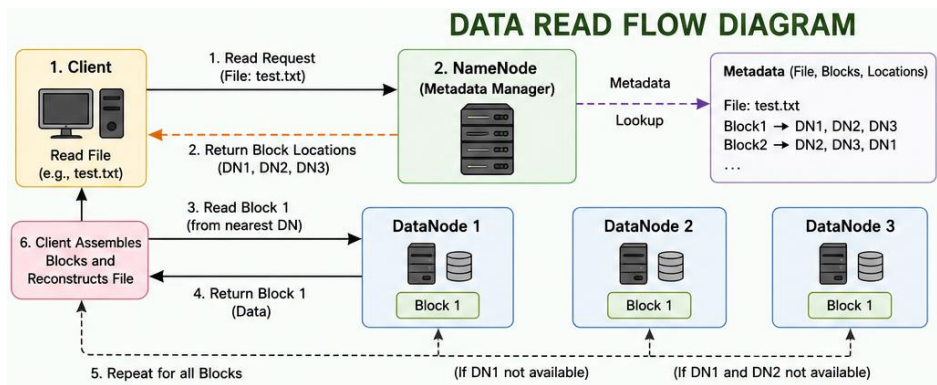
1. Client sends a file creation request to the NameNode.
2. NameNode verifies permissions and creates metadata.
3. NameNode provides DataNode locations.
4. File is split into blocks.
5. Client sends the first block to DataNode1.
6. DataNode1 forwards it to DataNode2.
7. DataNode2 forwards it to DataNode3.
8. Replication is completed.
9. Acknowledgments are returned to the client.
10. NameNode updates metadata.



Data Read Flow

When a client reads a file from HDFS, the following steps occur:

1. Client requests file information from the NameNode.
2. NameNode returns block locations.
3. Client connects to the nearest DataNode.
4. Data blocks are read.
5. Blocks are combined into the original file.
6. File is returned to the client.



Advantages of HDFS Data Flow

- High-speed data access.
- Fault-tolerant operation.
- Efficient block management.
- Supports large-scale distributed systems.
- Optimizes network utilization.

Module 3.2: Hadoop Architecture and HDFS

Contents

Hadoop I/O	1
Data Integrity	2
Compression.....	2
Serialization	3
Avro	3
File-Based Data Structures	4

Hadoop I/O

Data Input and Output

Hadoop I/O (Input/Output) refers to the mechanisms used by Hadoop to read data from storage, process it, and write the results back to storage. Efficient I/O operations are important for handling large-scale data processing tasks.

Features

- Supports reading and writing huge datasets.
- Works closely with HDFS and MapReduce.
- Provides various input and output formats.

- Optimized for distributed processing.

Importance

- Ensures efficient data access.
- Improves processing performance.
- Supports different data types and file formats.

Input Formats

InputFormat defines how input files are divided into Input Splits and how records are read for processing. **Functions**

- Splits input data into manageable chunks.
- Assigns data to Mapper tasks.
- Converts raw data into key-value pairs.

Types of Input Formats

1) TextInputFormat

- Default input format in Hadoop.
- Each line is treated as a record.
- Key = Byte offset.
- Value = Line content.

2) KeyValueTextInputFormat

- Splits each line into key and value pairs.
- Suitable for structured text data.

3) SequenceFileInputFormat

- Reads data stored in Sequence Files.
- Efficient for binary data.

4) NLineInputFormat

- Processes a fixed number of lines per split.
- Useful for customized data partitioning.

Output Formats

OutputFormat specifies how processed data is written to storage. **Functions**

- Writes MapReduce results.
- Determines output file structure.
- Handles output validation.

Types of Output Formats

1) TextOutputFormat

- Default output format.
- Stores output as text files.
- Produces human-readable results.

2) SequenceFileOutputFormat

- Writes output in binary Sequence File format.
- Suitable for large datasets.

3) MapFileOutputFormat

- Creates indexed output files.
- Supports faster searching.

Advantages

- Flexible storage options.
- Efficient handling of large outputs.
- Supports multiple file formats.

Data Integrity

Data Integrity refers to maintaining the accuracy, consistency, and reliability of data during storage, transfer, and processing. **Importance**

- Prevents data corruption.
- Ensures reliable results.
- Improves system trustworthiness.

Checksum Mechanism

A checksum is a calculated value used to verify that data has not been altered or corrupted. **Working**

1. Data is divided into blocks.
2. Hadoop generates a checksum for each block.
3. Checksums are stored separately.
4. During reading, a new checksum is calculated.
5. Both values are compared.

Benefits

- Detects corruption quickly.
- Ensures data reliability.
- Supports fault tolerance.

Error Detection

Error detection identifies damaged or altered data during storage or transmission. **Methods**

- Checksum verification.
- Block comparison.
- Replica validation.
- 4. Corrupted block is replaced automatically.

Advantages

- Maintains data accuracy.
- Reduces risk of data loss.
- Improves system reliability.

Process

1. Data is read from HDFS.
2. Checksum is verified.
3. If corruption is detected, Hadoop retrieves another replica.

Compression

Compression is the process of reducing the size of data files by encoding information more efficiently. **Need**

- Saves storage space.
- Reduces network traffic.
- Improves data transfer speed.
- Lowers storage costs.
- Enhances processing efficiency.

Types of Compression

- 1. Lossless Compression:** Data can be perfectly reconstructed after decompression. **Examples:** Gzip, Bzip2, Snappy, LZO. **Features**
 - No data loss.
 - Suitable for text and business data.
- 2. Lossy Compression:** Some data is permanently removed during compression. **Examples:** JPEG, MP3, MPEG. **Features**
 - Higher compression ratio.
 - Commonly used for multimedia files.

Common Hadoop Compression Codecs

Codec	Splittable	Speed
Gzip	No	Fast
Bzip2	Yes	Slow
Snappy	No	Very Fast
LZO	Optional	Fast

Advantages

- Reduces storage requirements.
- Improves data transmission speed.
- Minimizes disk I/O operations.
- Enhances Hadoop performance.
- Reduces infrastructure cost.

Serialization

Serialization is the process of converting an object into a stream of bytes so that it can be stored or transmitted across a network.

- Enables data exchange between systems.
- Supports distributed processing.
- Facilitates data storage.
- Data must be transferred efficiently.
- Objects need to be stored persistently.
- Improves processing performance.

Need for Serialization

- Hadoop nodes communicate through networks.

Benefits

- Faster communication.
- Reduced storage requirements.
- Platform-independent data transfer.

Types

1. Java Serialization

- Built into Java.
- Easy to use.
- Relatively slower.

2. Writable Serialization

- Hadoop's native serialization framework.
- Faster than Java Serialization.
- Optimized for MapReduce.

3. Avro Serialization

- Schema-based serialization.
- Supports multiple programming languages.
- Compact and efficient.

4. Protocol Buffers

- Developed by Google.
- High performance.
- Compact binary format.

Avro

Apache Avro is a data serialization system that stores both data and schema together, enabling efficient data exchange between applications. **Components**

- Schema
- Data File
- Serialization Engine

Features

- Schema-based storage.
- Compact binary format.
- Supports schema evolution.
- Language independent.
- Fast serialization and deserialization.
- Suitable for big data applications.

Schema Example

```
{
  "type": "record",
  "name": "Employee",
  "fields": [
    {"name": "id", "type": "int"},
    {"name": "name", "type": "string"}
  ]
}
```

Advantages

- Efficient storage.
- Faster processing.
- Supports multiple languages.
- Easy schema modification.
- Suitable for large datasets.

Applications

- Hadoop ecosystems.
- Data warehousing.
- Data exchange between systems.
- Stream processing.
- Big data analytics.

File-Based Data Structures

File-based data structures provide efficient methods for storing and accessing large datasets in Hadoop.

1) Sequence Files: A Sequence File is a flat binary file that stores data as key-value pairs.

Features

- Binary storage format.
- Highly efficient.
- Supports compression.
- Suitable for large datasets.

- Easy integration with MapReduce.

Applications

- Log storage.
- Intermediate MapReduce output.
- Large-scale data processing.

Advantages

- Faster processing.
- Reduced storage space.

2) Map Files: A Map File is an indexed Sequence File that enables fast data retrieval.

Components

1. Data File
2. Index File

Advantages

- Quick record retrieval.
- Improved query performance.
- Suitable for large datasets.

Features

- Key-value storage.
- Indexed access.
- Faster searches.
- Efficient lookups.

Applications

- Search systems.
- Lookup services.
- Metadata storage.

3) Array Files: An Array File stores values in an indexed array structure.

Features

- Sequential indexing.
- Efficient storage.
- Supports random access.

Advantages

- Fast retrieval using index.
- Easy data organization.
- Efficient processing.

Applications

- Numerical datasets.
- Scientific computing.
- Large collections of records.

4) Set Files: A Set File is a specialized Hadoop file structure used to store unique keys without duplicate entries.

Features

- Stores only unique elements.
- Supports membership testing.
- Built on Map Files.

Advantages

- Eliminates duplicates.
- Fast lookup operations.

- Efficient storage management.

Applications

- Unique record management.
- Duplicate detection systems.
- Membership verification.
- Data validation tasks.

Module 4: MapReduce & YARN Framework

Contents

MapReduce	1
MapReduce Architecture.....	2
Components of MapReduce.....	2
Anatomy of a MapReduce Job.....	6
Partitioning and Combining.....	7
Input and Output Formats	8
MapReduce Types	9
Testing MapReduce Applications	10
YARN Framework	11
Failures in MapReduce and YARN	13
Recovery Mechanisms.....	14
Job Scheduling	15
Shuffle and Sort	16

MapReduce

MapReduce is a distributed programming model and processing framework developed by Google and implemented in Hadoop for processing large volumes of data across multiple machines in a cluster. It divides a large task into smaller sub-tasks, processes them in parallel, and combines the results to produce the final output. **Key Idea**

- **Map** phase processes input data and generates intermediate key-value pairs.
- **Reduce** phase aggregates and summarizes the intermediate results.

Need for MapReduce

Traditional systems face difficulties when processing massive datasets due to limitations in storage, computation power, and scalability. MapReduce addresses these challenges by distributing processing across multiple nodes.

- To process very large datasets efficiently.
- To utilize multiple machines simultaneously.
- To reduce processing time through parallel execution.
- To provide fault tolerance in distributed environments.
- To handle structured, semi-structured, and unstructured data.

Example: Analyzing billions of website logs on a single computer may take days, whereas MapReduce can distribute the workload across hundreds of nodes and complete it much faster.

Features of MapReduce

1. **Distributed Processing:** Processes data across multiple nodes.
2. **Parallel Execution:** Executes tasks simultaneously.
3. **Scalability:** Easily expands by adding nodes.
4. **Fault Tolerance:** Automatically handles node failures.
5. **Data Locality:** Processes data near its storage location.

6. **Automatic Load Balancing:** Distributes workload evenly.
7. **Simple Programming Model:** Requires only Mapper and Reducer logic.

Advantages of MapReduce

1. **High Performance:** Reduces processing time through parallelism.
2. **Scalability:** Supports clusters of any size.
3. **Reliability:** Recovers automatically from failures.
4. **Cost Effectiveness:** Runs on inexpensive hardware.
5. **Flexibility:** Handles different types of data.
6. **Efficient Resource Utilization:** Optimizes cluster resources.
7. **Simplified Big Data Processing:** Hides distributed computing complexity.

MapReduce Architecture

MapReduce Architecture is the core processing framework of Hadoop that enables the distributed processing of large datasets across a cluster of computers. It divides a large job into smaller tasks, executes them in parallel on multiple nodes, and combines the results to produce the final output. The architecture provides scalability, fault tolerance, data locality, and efficient resource utilization, making it suitable for big data applications such as log analysis, search indexing, recommendation systems, and machine learning.

Components of MapReduce

The MapReduce framework follows a master-slave architecture in which a master node coordinates the execution of tasks and worker nodes perform the actual processing.

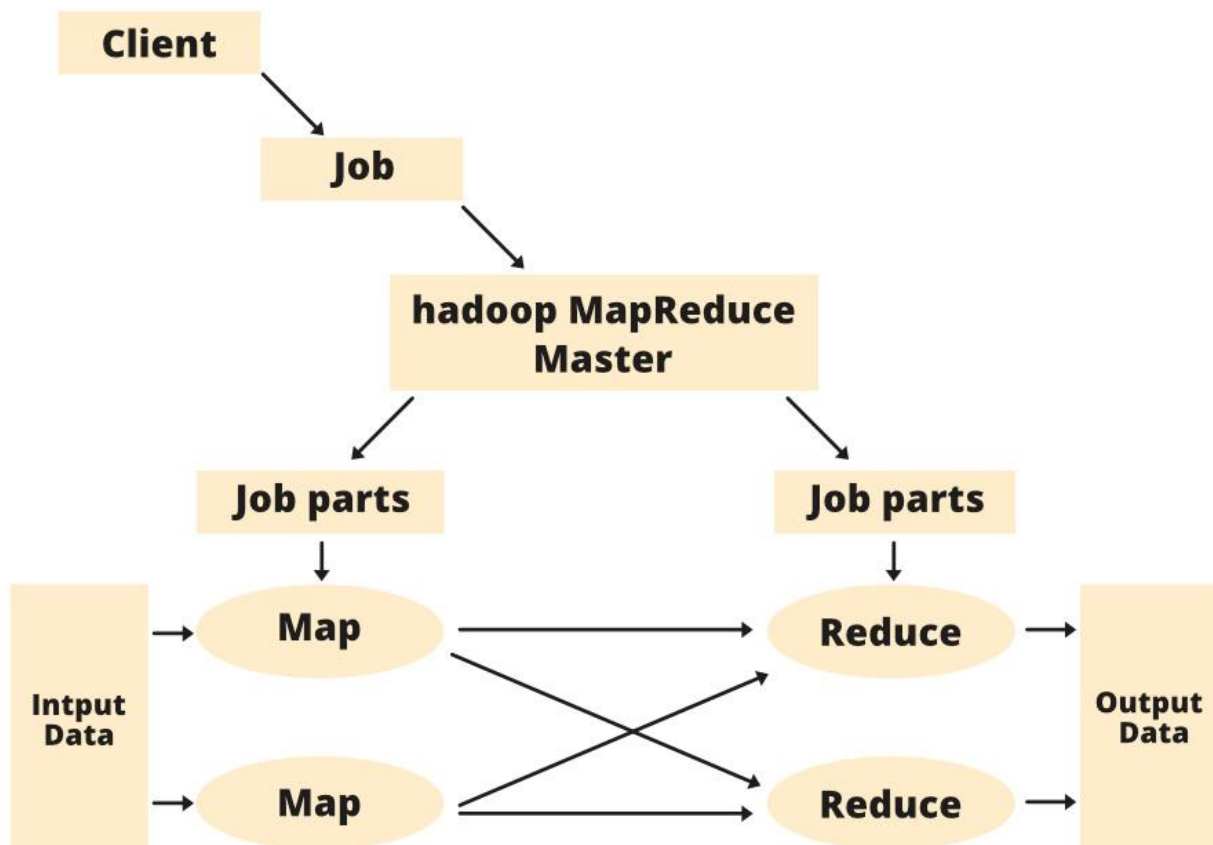
A. Core Components of MapReduce Framework

1. **Client:** The Client is the entry point to the MapReduce framework. It submits the job to Hadoop by specifying the input path, output path, and the Mapper and Reducer programs. After submitting the job, the client waits for the results.
2. **Job:** A Job represents the complete processing task submitted by the client. It contains all the information required for execution, including input data, output location, and processing logic. The job is divided into smaller tasks that are executed in parallel.
3. **Hadoop MapReduce Master:** The Master Node manages and coordinates the execution of MapReduce jobs. It accepts jobs from clients, divides them into tasks, assigns tasks to worker nodes, monitors their progress, and handles failures.
4. **Job Parts (Tasks):** Each job is divided into smaller units called tasks.
 - Map Tasks process input data and generate intermediate key-value pairs.
 - Reduce Tasks process intermediate data and generate the final output.

B. Processing Phases of MapReduce

1. **Map Phase:** The Map phase processes input data and converts it into intermediate key-value pairs. Each Mapper works independently on a portion of the input data.
2. **Shuffle and Sort Phase:** The intermediate output generated by Mappers is transferred to Reducers. During this phase, records with the same key are grouped together and sorted before being sent to Reducers.
3. **Reduce Phase:** The Reduce phase processes grouped key-value pairs and performs aggregation, summarization, or other computations to generate the final output.

Map Reduce Architecture



C. Processing Components

1. **Mapper:** A Mapper is responsible for processing input records and producing intermediate key-value pairs. **Functions of Mapper**

- Reads input data from HDFS.
- Processes each record independently.
- Generates intermediate key-value pairs.
- Supports parallel processing across multiple nodes.

Example

Input:

Big Data Big Hadoop

Mapper Output:

(Big,1)

(Data,1)

(Big,1)

(Hadoop,1)

2. **Reducer:** A Reducer processes the intermediate key-value pairs produced by Mappers and generates the final result. **Functions of Reducer**

- Receives grouped keys and values.
- Performs aggregation and calculations.
- Produces final output.
- Stores results in HDFS.

Example

Input to Reducer:

```
(Big,[1,1])  
(Data,[1])  
(Hadoop,[1])
```

Reducer Output:

```
(Big,2)  
(Data,1)  
(Hadoop,1)
```

D. Execution Management Components

1. **JobTracker and TaskTracker (Classic MapReduce):** In Hadoop 1.x, MapReduce used JobTracker and TaskTracker services.

a. **JobTracker:** is the master component responsible for managing MapReduce jobs.

- Accepts jobs from clients.
- Divides jobs into Map and Reduce tasks.
- Schedules tasks on worker nodes.
- Monitors task execution.
- Handles task failures and recovery.

b. **TaskTracker:** is the worker component that executes tasks assigned by the JobTracker.

- Executes Map and Reduce tasks.
- Reports task status to JobTracker.
- Stores intermediate results.
- Restarts failed tasks when required.

Limitations

- Single point of failure.
- Limited scalability.
- High workload on JobTracker.

2. **ResourceManager and NodeManager (YARN):** To overcome the limitations of the classic architecture, Hadoop introduced YARN (Yet Another Resource Negotiator).

a. **ResourceManager:** is the master component responsible for managing cluster resources.

- Allocates resources to applications.
- Schedules jobs across the cluster.
- Monitors resource usage.
- Coordinates application execution.

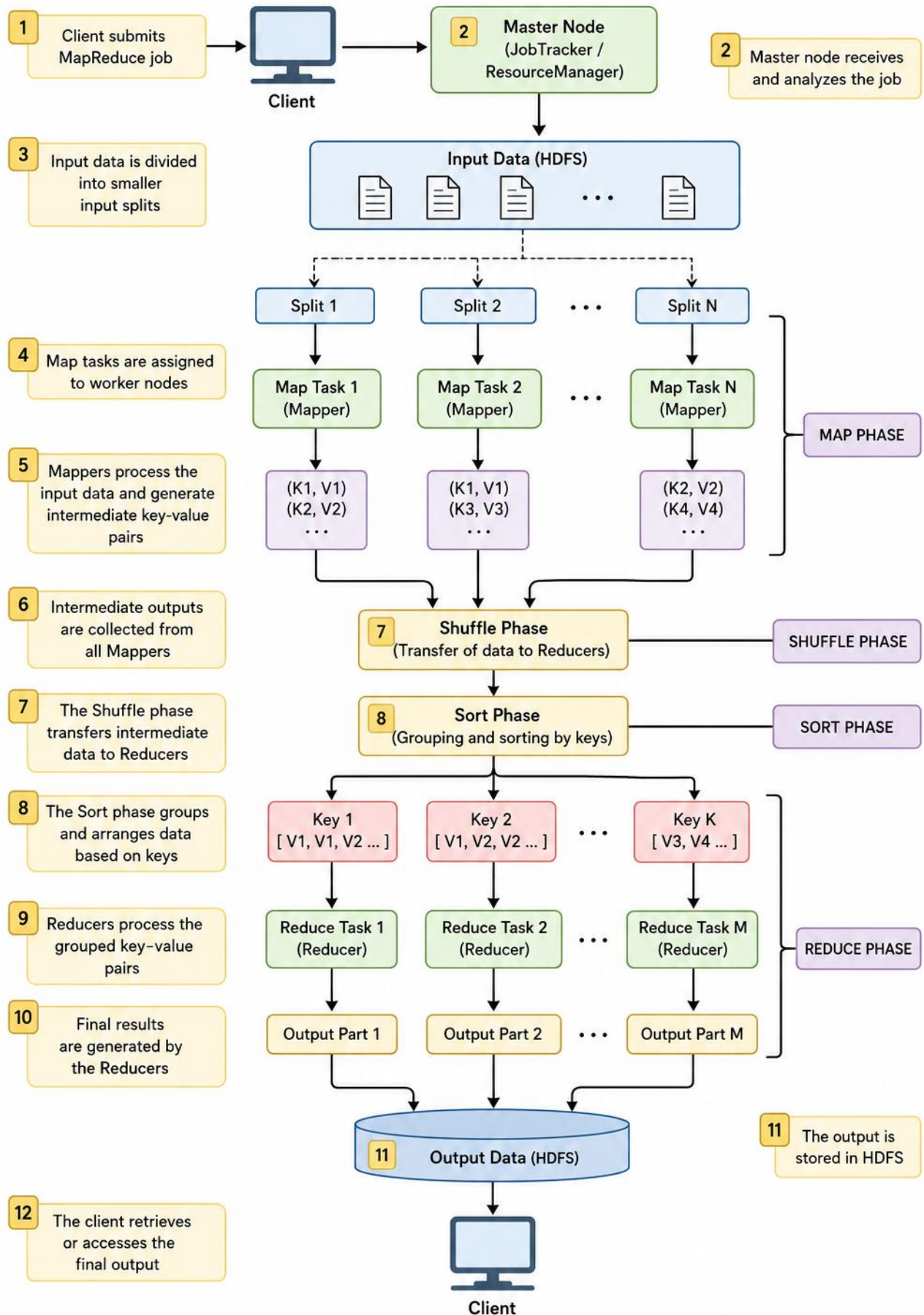
- Manages containers.
- Executes tasks assigned by ResourceManager.
- Monitors CPU, memory, and storage usage.
- Reports node status to ResourceManager.

b. **NodeManager:** is the worker component that runs on each cluster node.

Advantages of YARN

- Improved scalability.
- Better resource utilization.
- Supports multiple processing frameworks.
- Eliminates JobTracker bottleneck.

Execution Flow of MapReduce Architecture



Output Generation: Output Generation is the final stage where Reducer results are written to HDFS.

1. Reducer completes processing.
2. Results are stored in output files.
3. Files are saved in HDFS.

Features

- Distributed storage.
- Fault-tolerant output.
- Supports multiple output formats.

Anatomy of a MapReduce Job

1. **Job Submission:** is the first step in the execution of a MapReduce program. The client submits the job to the Hadoop cluster along with the Mapper, Reducer, configuration settings, and input/output paths. **Steps:**

1. Client prepares the MapReduce job.
2. Input and output paths are specified.
3. Mapper and Reducer classes are defined.
4. Job configuration is created.

5. Job is submitted to Hadoop for execution.

Importance:

- Initiates the MapReduce process.
- Provides all necessary execution details.

2. **Job Initialization:** After submission, Hadoop initializes the job and prepares it for execution. **Activities:**

1. Validates input and output paths.
2. Creates input splits from the input data.
3. Determines the number of Map and Reduce tasks.
4. Allocates cluster resources.

5. Schedules tasks for execution.

Importance:

- Ensures the job is ready for processing.
- Prevents errors before execution begins.

3. **Task Assignment:** is the process of distributing Map and Reduce tasks among available nodes in the cluster. **Process:**

1. Map tasks are assigned to worker nodes.
2. Data locality is considered while assigning tasks.
3. Reduce tasks are scheduled after Map tasks.

4. Workload is distributed across the cluster.

Importance:

- Improves performance.
- Balances resource utilization.

4. **Task Execution** During task execution, Mapper and Reducer functions process the data. **Steps:**

1. Map tasks read input splits.
2. Mappers generate intermediate key-value pairs.
3. Shuffle and Sort organize intermediate data.
4. Reducers process grouped data.

5. Final results are generated.

Importance:

- Performs actual data processing.
- Produces meaningful output from large datasets.

5. **Job Completion:** occurs when all tasks finish successfully and the output is stored. **Steps:**

1. Reducers write final results to HDFS.
2. Hadoop verifies task completion.

3. Resources are released.
4. Job status is marked as completed.
5. Client receives completion notification.

Importance:

- Indicates successful execution.
- Makes results available for use.

Partitioning and Combining

1. **Partitioning:** is the process of dividing intermediate key-value pairs among multiple Reducers.

Working:

- Hadoop uses the key to determine the target Reducer.
- Records with the same key always go to the same Reducer.

Formula:

Partition

$= \text{Hash}(\text{Key}) \% \text{Number of Reducers}$

Advantages:

- Balances workload among Reducers.
- Improves parallel processing.

Types of Partitioners

1. Hash Partitioner

- Default partitioner in Hadoop.
- Uses hash value of the key.
- Distributes data evenly.

2. Custom Partitioner

- User-defined partitioning logic.
- Used when specific key distribution is required.

3. Total Order Partitioner

- Produces globally sorted output.
- Useful for large-scale sorting applications.

2. **Combining:** A Combiner is an optional mini-Reducer that runs after the Map phase and before Shuffle and Sort.

Working:

1. Mapper generates intermediate data.
2. Combiner performs local aggregation.
3. Reduced data is sent to Reducers.

Importance: Reduces intermediate data transfer.

Example:

```
Mapper Output:

(Hadoop,1)
(Hadoop,1)
(Hadoop,1)

Combiner Output:

(Hadoop,3)
```

Benefits of Partitioning and Combining

Benefits of Partitioning

1. Balances workload among Reducers.
2. Improves scalability.
3. Enhances parallel processing.
4. Reduces execution time.

Benefits of Combining

1. Reduces network traffic.
2. Minimizes data transfer during Shuffle.
3. Improves job performance.
4. Saves cluster resources.

Input and Output Formats

Input and Output Formats define how data is read from input sources and written to output destinations in Hadoop MapReduce.

1. **Input Formats:** An Input Format in Hadoop MapReduce defines how input data is divided into input splits and how records are read before being processed by Mappers. It acts as an interface between the input data source and the Map phase.

Functions of Input Format

1. Divides input data into splits.
2. Creates a RecordReader.
3. Converts input data into key-value pairs.
4. Supplies records to Mapper tasks.

Common Input Formats

1. TextInputFormat

- Default input format in Hadoop.
- Reads data line by line.
- Key is the byte offset of the line.
- Value is the content of the line.
- Suitable for text files, log files, and CSV files.

2. KeyValueTextInputFormat

- Reads each line as a key-value pair.
- Key and value are separated by a delimiter (default: tab).
- Useful for structured text data.
- Commonly used in configuration and lookup files.

3. SequenceFileInputFormat

- Reads data stored in Hadoop SequenceFiles.
- Stores records as binary key-value pairs.
- Supports data compression.
- Provides faster processing for large datasets.

Advantages

1. Supports different data formats.
2. Enables efficient data processing.
3. Improves scalability and flexibility.

2. **Output Formats:** An Output Format in Hadoop MapReduce defines how the final output generated by Reducers is written and stored in the output destination such as HDFS.

Functions of Output Format

1. Writes final key-value pairs.
2. Creates output files.
3. Manages output storage.
4. Determines output data format.

Common Output Formats

1. TextOutputFormat

- Default output format in Hadoop.
- Writes output as plain text.
- Stores one key-value pair per line.

- Easy to read and suitable for reports and analysis.

2. SequenceFileOutputFormat

- Writes output in SequenceFile format.
- Stores data as binary key-value pairs.
- Supports compression and efficient storage.
- Commonly used as input for subsequent MapReduce jobs.

Advantages

1. Supports multiple output formats.
2. Provides efficient data storage.

3. Facilitates further processing and analysis.

4. Improves output management in Hadoop.

MapReduce Types

MapReduce jobs can be classified based on the use of Mapper and Reducer functions. Different types are used depending on the processing requirements.

1. **Mapper-Only Jobs:** A Mapper-Only Job uses only the Mapper phase and does not require any Reducer.

Working

1. Input data is processed by Mappers.
2. Mapper generates the final output directly.
3. Output is written to HDFS.

Features

- No Reduce phase.
- Faster execution.
- Suitable for filtering and transformation tasks.

Applications

- Data cleaning.
- Log filtering.
- Format conversion.

2. **Reducer-Only Jobs:** A Reducer-Only Job uses only the Reducer phase with no Mapper processing.

Working

1. Input data is directly provided to Reducers.
2. Reducers process and aggregate the data.
3. Final output is generated.

Features

- No Map phase.
- Used when only aggregation is required.
- Less common in practice.

Applications

- Data summarization.
- Simple aggregation operations.

3. **Map-Reduce Jobs:** A Map-Reduce Job uses both Mapper and Reducer phases and

is the most common type of MapReduce job.

Working

1. Mapper processes input data.
2. Intermediate key-value pairs are generated.
3. Shuffle and Sort organize the data.
4. Reducers aggregate the results.
5. Final output is stored in HDFS.

Features

- Supports large-scale data processing.
- Provides parallel execution.
- Highly scalable and fault tolerant.

Applications

- Word Count.
- Log Analysis.
- Data Mining.
- Recommendation Systems.

4. **Chained MapReduce Jobs:** A Chained MapReduce Job consists of multiple MapReduce jobs executed sequentially, where the output of one job becomes the input of the next job.

Working

1. First MapReduce job processes the input.
2. Output is stored in HDFS.
3. Next MapReduce job uses this output as input.
4. The process continues until the final result is obtained.

Features

- Supports complex workflows.
- Breaks large problems into smaller tasks.

- Improves modularity and maintainability.

Applications

- Machine Learning Pipelines.

- Data Warehousing.
- ETL (Extract, Transform, Load) Processes.
- Large-Scale Analytics.

Testing MapReduce Applications

Testing ensures that MapReduce programs produce correct results before deployment on a Hadoop cluster.

1. **MRUnit:** MRUnit is a testing framework developed for Hadoop MapReduce applications. It allows developers to test Mapper and Reducer logic without running a complete Hadoop cluster.

- Tests Mapper, Reducer, and Driver classes.
- Simplifies debugging and validation.
- Reduces development time.
- Runs locally on a single machine.

2. **Unit Testing with MRUnit:** Unit testing verifies the correctness of individual Mapper and Reducer functions.

Process

- Create test input data.
- Execute Mapper or Reducer using MRUnit.
- Compare actual output with expected output.

- Identify and fix errors if necessary.

Benefits

- Improves code reliability.
- Detects bugs early.
- Ensures accurate processing.

3. **Test Data Preparation:** Test data consists of sample input records used to verify MapReduce logic.

Guidelines

- Use representative sample data.
- Include normal and edge-case inputs.
- Prepare expected output for comparison.

- Keep datasets small for quick testing.

Benefits

- Improves test accuracy.
- Helps identify logical errors.
- Simplifies debugging.

4. **Local Testing:** Local testing executes MapReduce jobs on a single machine without a Hadoop cluster.

Process

- Configure Hadoop in local mode.
- Run MapReduce application.
- Verify output results.
- Debug and optimize code.

Advantages

- Faster execution.
- No cluster required.
- Easier debugging and development.

YARN Framework

YARN (Yet Another Resource Negotiator) is the resource management layer of Hadoop introduced in Hadoop 2. It separates resource management from data processing, allowing multiple applications such as MapReduce, Spark, Hive, and Tez to run on the same Hadoop cluster. YARN improves scalability, cluster utilization, and overall performance.

Objectives of YARN

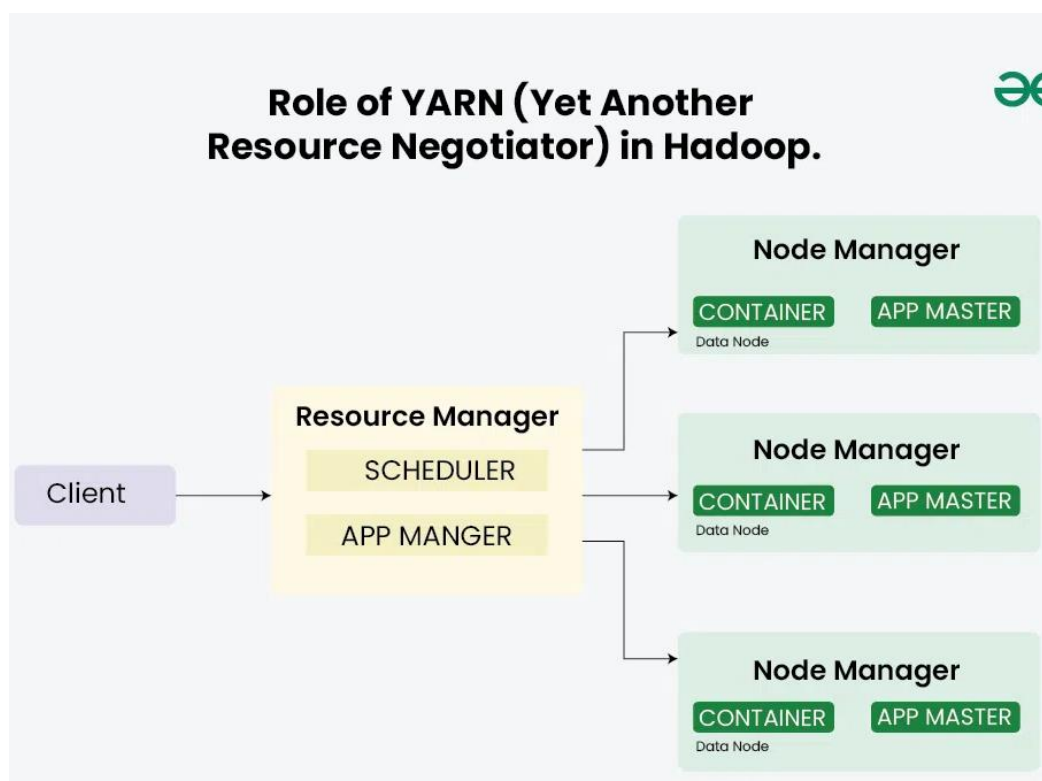
1. Efficient management of cluster resources.
2. Support for multiple data processing frameworks.
3. Improved scalability for large clusters.
4. Better utilization of hardware resources.
5. Simplified job scheduling and monitoring.

Features of YARN

1. Dynamic resource allocation.
2. Multi-tenancy support.
3. High scalability.
4. Fault tolerance.
5. Better cluster utilization.
6. Supports distributed computing applications.
7. Flexible scheduling mechanisms.

Architecture of YARN

YARN follows a master-slave architecture consisting of four main components:



1. ResourceManager (RM)

- Master daemon of YARN.
- Manages cluster-wide resources.
- Allocates resources to applications.
- Performs job scheduling and monitoring.

2. NodeManager (NM)

- Runs on every worker node.
- Manages resources on the local machine.
- Launches and monitors containers.
- Reports node status to ResourceManager.

3. ApplicationMaster (AM)

- Created separately for each application.
- Negotiates resources with ResourceManager.
- Coordinates execution of tasks.
- Monitors application progress and failures.

Advantages of YARN

1. Separates resource management from processing.
2. Supports multiple frameworks beyond MapReduce.
3. Improves scalability and flexibility.
4. Increases cluster utilization.
5. Provides fault tolerance and reliability.
6. Enables efficient resource sharing among applications.

Working of YARN

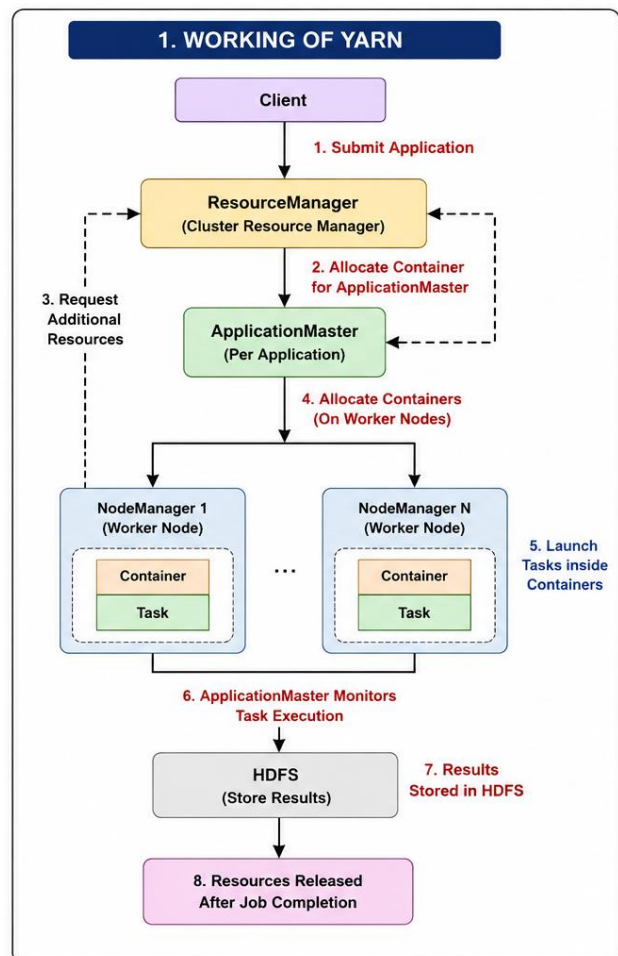
1. Client submits an application to ResourceManager.
2. ResourceManager allocates a container for ApplicationMaster.
3. ApplicationMaster starts and requests additional resources.
4. ResourceManager allocates containers on worker nodes.
5. NodeManagers launch tasks inside containers.
6. ApplicationMaster monitors task execution.
7. Results are generated and stored in HDFS.
8. Resources are released after job completion.

4. Container

- Smallest unit of resource allocation in YARN.
- Contains CPU, memory, disk, and network resources.
- Executes Mapper, Reducer, or other application tasks.

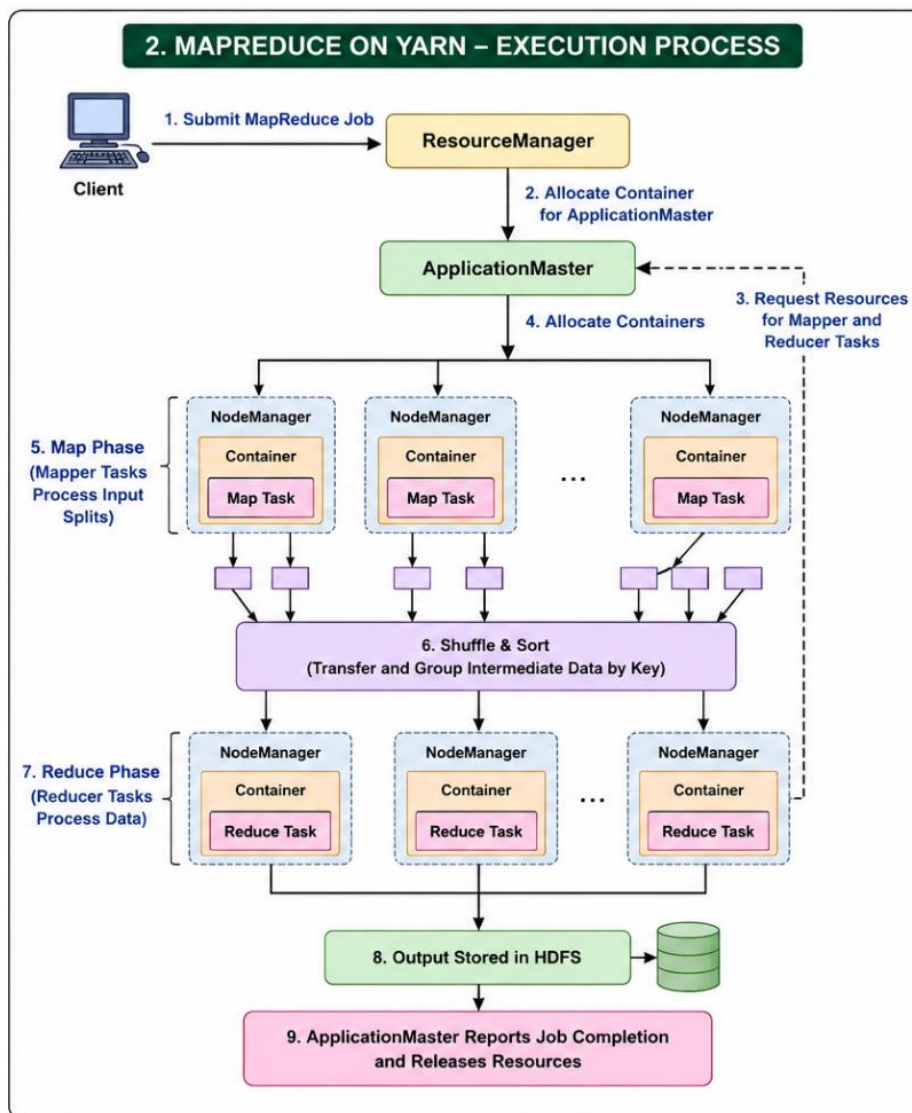
Limitations of YARN

1. Configuration can be complex.
2. Requires additional memory for management services.
3. Monitoring large clusters may be challenging.
4. Performance depends on proper resource allocation.



Execution Process for MapReduce on YARN

1. The client submits a MapReduce job to the ResourceManager.
2. ResourceManager allocates a container for the ApplicationMaster.
3. ApplicationMaster starts and requests resources for Mapper and Reducer tasks.
4. ResourceManager allocates containers on available nodes.
5. NodeManagers launch Mapper and Reducer tasks within containers.
6. Mappers process input data and generate intermediate output.
7. Shuffle and Sort transfer intermediate data to Reducers.
8. Reducers process the data and generate final output.
9. Output is stored in HDFS.
10. ApplicationMaster reports job completion and releases resources.



Failures in MapReduce and YARN

Failures are common in large distributed systems. Hadoop provides mechanisms to detect failures and recover automatically.

1. **Task Failure:** occurs when a Mapper or Reducer task cannot complete successfully.
1. Hardware malfunction.

2. Software bugs.
3. Insufficient memory.
4. Network issues.

Handling

1. Failed tasks are automatically restarted.
2. Tasks may be executed on another node.

2. **Node Failure:** occurs when an entire worker node becomes unavailable.

Causes

1. Power failure.
2. Hardware crash.
3. Operating system failure.
4. Network disconnection.

Handling

1. Heartbeat messages detect node failure.
2. Tasks are reassigned to healthy nodes.
3. HDFS replicas ensure data availability.

3. **Job Failure:** occurs when a complete MapReduce job cannot finish successfully.

Causes

1. Repeated task failures.
2. Application errors.
3. Resource shortages.
4. Configuration problems.

Effects

1. Job execution stops.
2. Error information is logged for analysis.

Recovery Mechanisms

1. **Recovery Mechanisms in MapReduce:** MapReduce includes built-in fault tolerance mechanisms.

Recovery Methods

1. Automatic task re-execution.
2. Data replication through HDFS.
3. Speculative execution for slow tasks.
4. Task reassignment to healthy nodes.

Benefits

1. Improved reliability.
2. Continuous processing despite failures.
3. Reduced data loss.

2. **Recovery Mechanisms in YARN:** YARN provides advanced failure recovery mechanisms.

Recovery Methods

1. ResourceManager monitors cluster health.
2. NodeManager detects local failures.
3. ApplicationMaster can be restarted.
4. Failed containers are recreated automatically.
5. Tasks are rescheduled on available nodes.

Benefits

1. Faster recovery.
2. Better resource utilization.
3. Improved application availability.

Job Scheduling

Job scheduling determines how resources are allocated among multiple jobs in a Hadoop cluster. **Need for Scheduling:** Scheduling ensures efficient utilization of cluster resources and fair execution of jobs.

Objectives

1. Maximize resource utilization.
2. Reduce job waiting time.
3. Improve cluster performance.
4. Ensure fairness among users.

4. **FIFO Scheduler:** FIFO (First In First Out) executes jobs in the order they are submitted.

Features

- Simplest scheduling algorithm.
- Jobs run sequentially based on arrival time.
- Older jobs receive priority.

Advantages

- Easy to implement.
- Low scheduling overhead.

Disadvantages

- Long jobs can delay short jobs.
- Poor resource sharing.

5. **Capacity Scheduler:** Capacity Scheduler divides cluster resources into multiple queues.

Features

- Each queue receives a fixed resource capacity.

- Supports multiple organizations or departments.
- Unused resources can be shared temporarily.

Advantages

- Better resource utilization.
- Supports multi-user environments.
- Prevents resource monopolization.

6. **Fair Scheduler:** Fair Scheduler allocates resources so that all jobs receive a fair share over time.

Features

- Jobs run concurrently.
- Resources are distributed fairly.
- Small jobs finish quickly.

Advantages

- Reduced waiting time.
- Better responsiveness.
- Improved cluster efficiency.

Feature	FIFO	Capacity	Fair
Execution Order	Submission Order	Queue-Based	Fair Sharing
Multi-User Support	No	Yes	Yes
Resource Sharing	Limited	Good	Excellent
Short Job Performance	Poor	Moderate	Good
Complexity	Low	Medium	High

Shuffle and Sort

Shuffle and Sort is the intermediate phase between Mapper and Reducer. It organizes Mapper output before Reducer processing.

7. **Shuffle Process:** transfers intermediate key-value pairs from Mappers to appropriate Reducers.

Steps

1. Mapper generates intermediate data.
2. Data is partitioned based on keys.
3. Intermediate data is transferred across the network.
4. Reducers receive relevant records.

Functions

1. Data transfer.
2. Data grouping.
3. Key distribution.

Importance of Shuffle and Sort

1. Groups records with the same key.
2. Ensures correct reducer input.
3. Improves data organization.

8. **Sorting Process:** Sorting arranges intermediate data by key before it reaches the Reducer.

Steps

1. Received records are grouped by key.
2. Keys are sorted automatically.
3. Sorted data is passed to Reducers.

Benefits

1. Efficient aggregation.
2. Faster reducer execution.
3. Organized data processing.

4. Enables efficient aggregation and analysis.
5. Acts as a bridge between Map and Reduce phases.

Module 5.1: HBase

Contents

Distributed Databases.....	1
Distributed Databases in Big Data	2
HBase	2
Architecture of HBase	3
HBase Data Model.....	4
HBase Implementation.....	4
HBase Clients	4
Practical Applications of HBase	5

Distributed Databases

A distributed database is a database whose storage devices are not attached to a common processing unit but are distributed across a network. It is a collection of logically related data that is physically stored at different locations and managed by a Distributed Database Management System (DDBMS). Distributed databases are designed to improve scalability, reliability, availability, and performance in large-scale applications. In modern computing environments, organizations generate massive amounts of data that cannot be efficiently handled by a single database server. Distributed databases solve this problem by distributing data across multiple nodes while allowing users to access and manage data transparently.

1. Data is stored on multiple machines.
2. Users view the database as a single system.
3. Data can be accessed from different locations.
4. Supports data sharing and collaboration.
5. Managed by a Distributed Database Management System (DDBMS).

Example: A multinational company may store employee records in different countries while allowing centralized access through a distributed database system.

Characteristics of Distributed Databases

- **Data Distribution** – Data is stored across multiple locations or nodes.
- **Transparency** – Users access data without knowing its physical location.
- **Scalability** – The system can grow by adding more nodes.
- **Reliability** – Data remains available even if some nodes fail.
- **Concurrency** – Multiple users can access and modify data simultaneously.
- **Autonomy** – Each site can operate independently while remaining connected.

Advantages of Distributed Databases

- **High Availability** – Services remain operational despite node failures.
- **Improved Performance** – Data can be processed closer to where it is stored.
- **Scalability** – Additional nodes can be added to handle increased workloads.
- **Fault Tolerance** – Data replication helps prevent data loss.
- **Reduced Communication Cost** – Local access reduces network traffic.
- **Flexibility** – Supports geographically distributed users and applications.

Challenges of Distributed Databases

- **Data Consistency** – Keeping replicated data synchronized is difficult.
- **Complexity** – Management and administration are more complicated.
- **Security Issues** – Protecting data across multiple locations is challenging.
- **Network Dependence** – Performance depends on network reliability.
- **Transaction Management** – Coordinating transactions across nodes is complex.
- **Higher Cost** – Additional hardware, software, and maintenance may be required.

Distributed Databases in Big Data

Big Data applications generate enormous volumes of structured, semi-structured, and unstructured data. Traditional databases often struggle to manage such massive datasets. Distributed databases play a crucial role in Big Data environments by providing scalable and fault-tolerant storage solutions.

Role in Big Data

1. Store petabytes of data across multiple nodes.
2. Support parallel data processing.
3. Provide high availability and fault tolerance.
4. Enable real-time and batch analytics.
5. Handle rapidly growing datasets efficiently.

Popular Distributed Databases in Big Data

Database	Type
HBase	Column-Oriented NoSQL Database
Cassandra	Distributed Column-Family Database
MongoDB	Document-Oriented Database
Couchbase	Distributed NoSQL Database

Applications

1. Social media platforms.
2. E-commerce websites.
3. Banking and financial systems.
4. Healthcare data management.
5. IoT and sensor networks.
6. Cloud computing environments.

Importance: Distributed databases form the foundation of modern Big Data ecosystems such as Hadoop and Spark. They provide scalable storage, fault tolerance, and high-speed data access required for processing massive datasets efficiently.

HBase

HBase (Hadoop Database) is an open-source, distributed, column-oriented NoSQL database built on top of HDFS. It is designed to store and manage large volumes of structured and semi-structured data across clusters of commodity hardware. HBase provides real-time read and write access to big data and is modeled after Google's Bigtable.

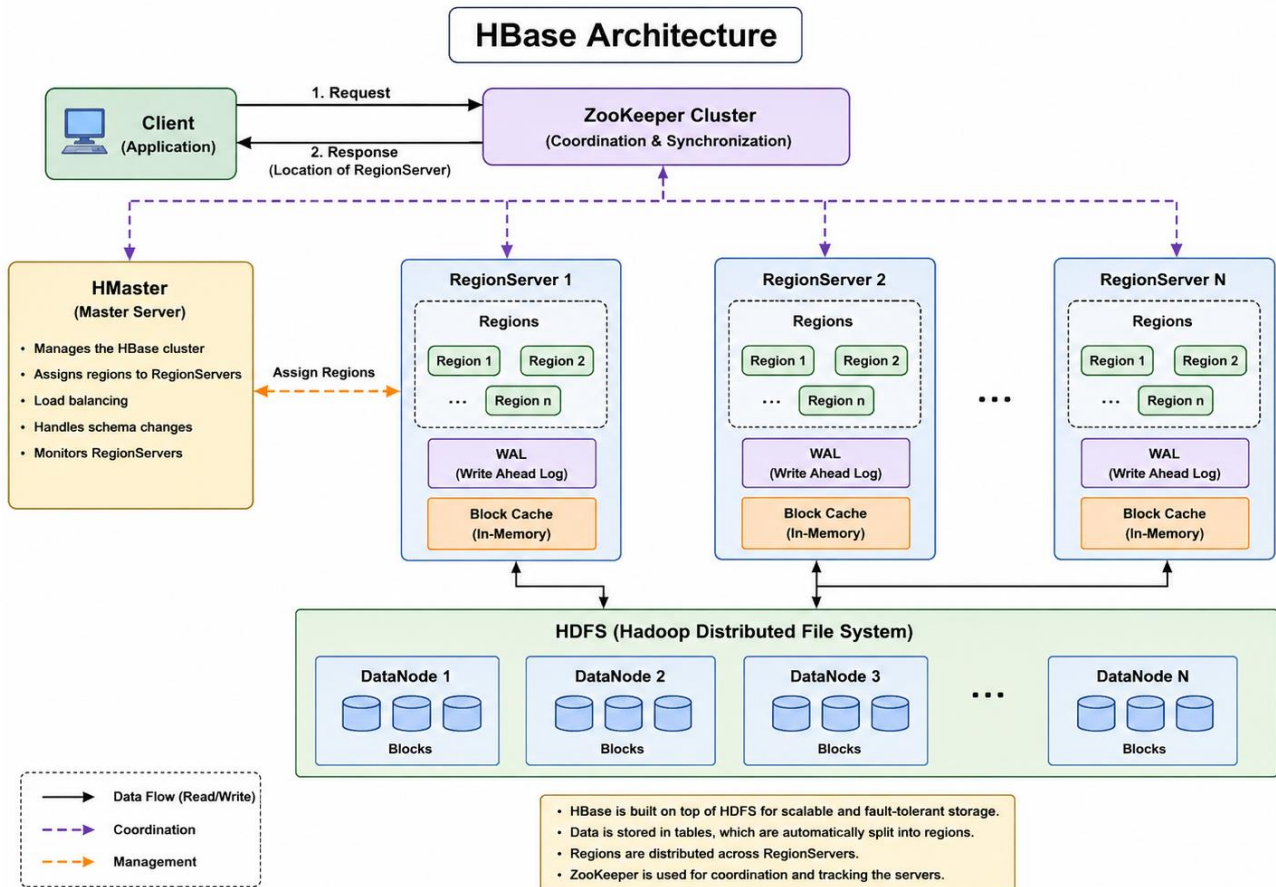
- Runs on top of HDFS.
- Supports random, real-time read/write operations.
- Designed for massive scalability.
- Suitable for sparse and large datasets.
- Integrates well with Hadoop ecosystem tools.

Features of HBase

- **Distributed Storage** – Data is stored across multiple nodes in a cluster.
- **Scalability** – Can scale horizontally by adding more servers.
- **Fault Tolerance** – Uses HDFS replication for data reliability.
- **Real-Time Access** – Supports fast read and write operations.
- **Column-Oriented Storage** – Data is stored by columns instead of rows.
- **Strong Consistency** – Ensures consistent data reads and writes.
- **Flexible Schema** – Columns can be added dynamically.
- **Integration with Hadoop** – Works seamlessly with HDFS, MapReduce, and YARN.

Architecture of HBase

HBase follows a master-slave architecture where a central HMaster coordinates multiple RegionServers. Data is stored in HDFS, while ZooKeeper manages coordination and monitoring.



- 1. HMaster:** HMaster is the master server responsible for managing and coordinating the HBase cluster.
 - Assigns regions to RegionServers.
 - Performs load balancing.
 - Handles schema changes.
 - Monitors RegionServer status.
 - Recovers from server failures.
- 2. RegionServer:** RegionServer stores and manages table regions and serves client requests.
 - Handles read and write operations.
 - Manages table regions.
 - Communicates with HDFS.
 - Splits large regions automatically.
 - Maintains in-memory cache for faster access.
- 3. ZooKeeper:** ZooKeeper is a distributed coordination service used by HBase.
 - Tracks active servers.
 - Maintains cluster configuration.
 - Helps clients locate RegionServers.
 - Detects failures.
 - Coordinates distributed operations.
- 4. HDFS:** HDFS is the storage layer of HBase.
 - Stores HBase data files.
 - Provides fault tolerance through replication.
 - Ensures high availability.
 - Supports distributed storage.
 - Handles large-scale data efficiently.

Working

1. Client sends request to HBase.
2. ZooKeeper identifies the appropriate RegionServer.

3. RegionServer processes the request.
4. Data is stored or retrieved from HDFS.
5. HMaster manages load balancing and region assignments.

HBase Data Model

HBase uses a column-oriented data model based on Google's Bigtable. Data is organized into tables, rows, column families, columns, and cells. **Components of HBase Data Model**

1) Table

- A collection of related data.
- Similar to a table in a relational database.
- Consists of rows and column families.

2) Row Key

- Unique identifier for each row.
- Rows are stored in sorted order based on row keys.
- Used for fast data retrieval.

3) Column Family

- Groups related columns together.
- Defined during table creation.

- Physical storage unit in HBase.

4) Column Qualifier

- Specific column within a column family.
- Can be added dynamically.

5) Cell

- Intersection of a row and column.
- Stores the actual data value.

6) Timestamp

- Each cell contains a timestamp.
- Allows storage of multiple versions of data.

Table: Student		
Row Key	Column Family: Info	Column Family: Marks
101	Name = John	Java = 85
	Age = 20	DBMS = 90
102	Name = Alice	Java = 88
	Age = 21	DBMS = 92

HBase Implementation

HBase is implemented on top of HDFS and follows a distributed architecture.

Working

1. Data is divided into regions.
2. Regions are distributed across RegionServers.
3. RegionServers store data in HDFS.
4. HMaster manages the cluster.
5. ZooKeeper coordinates communication.

6. Clients access data through RegionServers.

Advantages of Implementation

- Horizontal scalability.
- High availability.
- Fault tolerance.
- Real-time data access.
- Efficient handling of large datasets.

HBase Clients

HBase clients are applications or APIs used to interact with HBase tables. **Types of HBase Clients**

1) Java API Client

- Most commonly used client.
- Provides direct access to HBase operations.
- Supports CRUD operations.

2) HBase Shell

- Command-line interface.
- Used for administration and testing.
- Supports table creation and data manipulation.

3) REST Client

- Accesses HBase through REST APIs.
- Suitable for web applications.

4) Thrift Client

- Provides cross-language access.
- Supports Python, C++, PHP, Ruby, etc.

5) MapReduce Client

- Integrates HBase with Hadoop MapReduce.
- Used for large-scale data processing.

Practical Applications of HBase

1. **Social Media** – Stores user profiles, posts, comments, and activity logs.
2. **E-Commerce** – Manages product catalogs, customer information, and purchase history.
3. **IoT Systems** – Stores continuous streams of sensor-generated data.
4. **Banking and Finance** – Maintains transaction records and customer account data.
5. **Healthcare** – Stores patient records, prescriptions, and medical histories.
6. **Telecommunications** – Manages call records and network usage data.
7. **Web Analytics** – Tracks website traffic, user behavior, and clickstream data.
8. **Recommendation Systems** – Stores user preferences for personalized recommendations.
9. **Fraud Detection** – Processes large datasets to identify suspicious activities.
10. **Online Gaming** – Maintains player profiles, scores, and game statistics.

Module 5.2: Cassandra

Contents

Cassandra	1
Cassandra Data Model	2
Cassandra Clients.....	3
HBase vs Cassandra	3

Cassandra

Apache Cassandra is an open-source, distributed NoSQL database designed to handle large amounts of data across multiple servers without a single point of failure. It provides high availability, fault tolerance, and horizontal scalability, making it suitable for big data and real-time applications.

- Developed originally by Facebook.
- Distributed and decentralized database.
- Supports large-scale data storage.
- Provides high availability and fault tolerance.
- Suitable for real-time applications.

Features of Cassandra

- **Distributed Architecture** – Data is distributed across multiple nodes.
- **High Availability** – No single point of failure in the system.
- **Fault Tolerance** – Data remains available even during node failures.
- **Linear Scalability** – Capacity increases by adding more nodes.
- **Peer-to-Peer Design** – All nodes have equal responsibilities.
- **Automatic Replication** – Data is replicated across multiple nodes.
- **Fast Write Performance** – Optimized for high-speed data insertion.
- **Flexible Schema** – Supports dynamic and evolving data structures.
- **Tunable Consistency** – Consistency level can be adjusted as needed.
- **Multi-Datacenter Support** – Supports deployment across multiple locations.

Cassandra Architecture

Cassandra follows a distributed peer-to-peer architecture where all nodes are equal and communicate directly with each other. There is no master server.

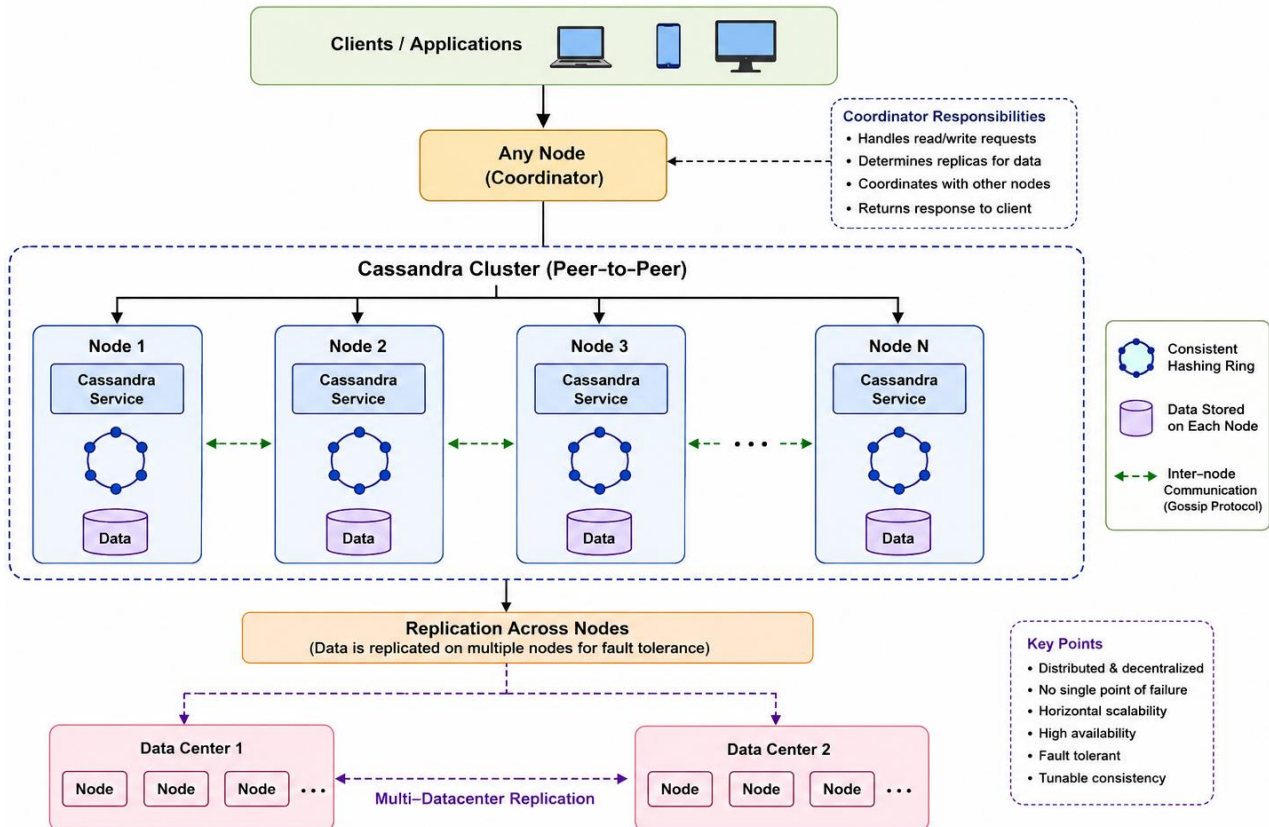
Working

1. Client sends a request to any node.
2. The receiving node acts as the coordinator.
3. Data is distributed across nodes using partitioning.
4. Replicas store copies of the data.
5. Nodes communicate to maintain consistency.
6. Requested data is returned to the client.

Key Elements of Cassandra Architecture

- **Client/Application** – Sends read and write requests to the Cassandra cluster.
- **Coordinator Node** – Receives client requests and coordinates communication with other nodes.
- **Cluster** – A collection of interconnected Cassandra nodes.
- **Nodes** – Individual servers that store and manage data.
- **Partitioning** – Distributes data across nodes using a partition key.
- **Replication** – Stores copies of data on multiple nodes for reliability.
- **Gossip Protocol** – Enables nodes to exchange status and cluster information.
- **Data Centers** – Groups of nodes that can be distributed across different geographic locations.

CASSANDRA ARCHITECTURE

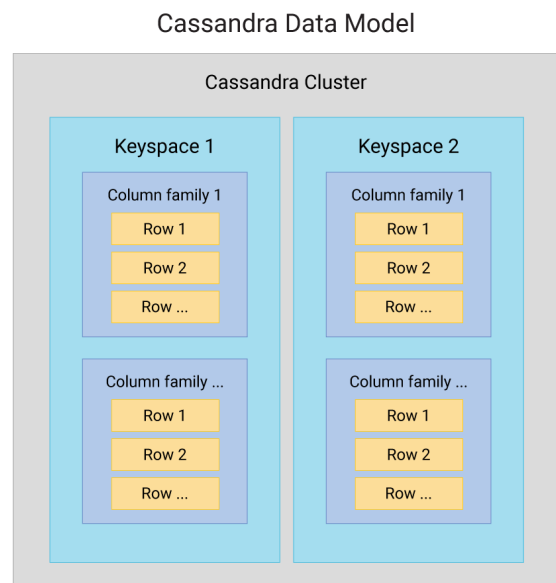


Advantages of Cassandra Architecture

- No single point of failure.
- High availability and fault tolerance.
- Horizontal scalability by adding more nodes.
- Fast read and write performance.
- Supports multi-data-center deployments.

Cassandra Data Model

- **Keyspace** – The top-level container in Cassandra, similar to a database in RDBMS.
- **Table (Column Family)** – Stores related data in rows and columns.
- **Row** – A collection of columns identified by a unique primary key.
- **Column** – Basic unit of data containing a name, value, and timestamp.
- **Primary Key** – Uniquely identifies rows and determines data distribution.
- **Partition Key** – Decides the node where data is stored.
- **Clustering Column** – Defines the sorting order of data within a partition.
- **Wide Rows** – Rows that can contain a large number of columns.
- **Schema-Free Design** – Allows flexible handling of large and evolving datasets.



Cassandra Clients

1) CQL Shell (cqlsh)

- Command-line interface used to interact with Cassandra.
- Supports creating keyspaces, tables, and executing queries.
- Similar to SQL shells in relational databases.
- Useful for database administration and testing.

2) Java Driver

- Official driver for connecting Java applications to Cassandra.
- Provides high-performance query execution.
- Supports asynchronous operations and connection pooling.
- Widely used in enterprise applications.

3) Python Driver

- Allows Python applications to communicate with Cassandra.
- Supports CQL query execution and data retrieval.
- Easy integration with data analytics and machine learning projects.
- Commonly used in scripting and automation tasks.

4) Other Language Drivers

- Node.js Driver for JavaScript applications.
- C# Driver for .NET applications.
- Go Driver for Go-based systems.
- PHP and Ruby Drivers for web application development.

HBase vs Cassandra

Feature	HBase	Cassandra
Developer	Apache Software Foundation	Apache Software Foundation (originally Facebook)
Database Type	Column-oriented NoSQL database	Column-family NoSQL database
Architecture	Master-Slave Architecture	Peer-to-Peer Architecture
Master Node	Uses HMaster	No master node
Storage	Runs on HDFS	Uses its own distributed storage
Scalability	Horizontal scalability	Horizontal scalability
Fault Tolerance	Achieved through HDFS replication	Achieved through data replication across nodes
Read Performance	Good	Very high
Write Performance	High	Very high
Consistency	Strong consistency	Tunable consistency
Availability	High	Very high
Data Distribution	Regions managed by Region Servers	Partitioning using consistent hashing
Query Language	HBase Shell, Java API	CQL (Cassandra Query Language)
Hadoop Integration	Native integration with Hadoop	Integration through connectors
Best Use Cases	Batch processing, analytics, Hadoop ecosystems	Real-time applications, high-availability systems

Module 6.1: Pig

Contents

Introduction to High-Level Data Processing Tools.....	1
Apache Pig	1
Grunt.....	3
Pig Data Model	4
Pig Latin.....	5

Introduction to High-Level Data Processing Tools

High-Level Data Processing Tools are software frameworks that simplify the processing and analysis of large datasets in Hadoop. Instead of writing complex MapReduce programs, users can use tools such as Pig and Hive to perform data processing through simple scripts and query languages. These tools improve productivity, reduce development time, and make big data analytics accessible to users with limited programming knowledge.

Need for High-Level Data Processing Tools

- Simplify big data processing tasks in Hadoop.
 - Reduce the complexity of writing MapReduce programs.
 - Enable faster development and execution of data processing jobs.
 - Provide user-friendly scripting and query languages.
 - Allow non-programmers and analysts to work with large datasets.
 - Improve productivity and reduce coding effort.
 - Support data transformation, analysis, and reporting efficiently.
- ### Limitations of MapReduce
- Requires extensive Java programming knowledge.
 - Writing MapReduce code is time-consuming and complex.
 - Large amount of code is needed for simple operations.
 - Difficult to debug and maintain programs.
 - Not suitable for interactive querying.
 - Development cycle is slower compared to high-level tools.
 - Complex workflows require multiple MapReduce jobs.
 - Limited support for ad-hoc data analysis.

Apache Pig

Apache Pig is a high-level platform for processing and analyzing large datasets in the Hadoop ecosystem. It provides a scripting language called **Pig Latin**, which simplifies the development of data processing applications. Pig automatically translates Pig Latin scripts into MapReduce, Tez, or Spark jobs for execution on Hadoop clusters.

- Developed by Yahoo and later contributed to Apache.
- Simplifies complex Hadoop data processing tasks.
- Uses Pig Latin scripting language.
- Suitable for ETL (Extract, Transform, Load) operations.
- Works efficiently with structured, semi-structured, and unstructured data.

Features of Pig

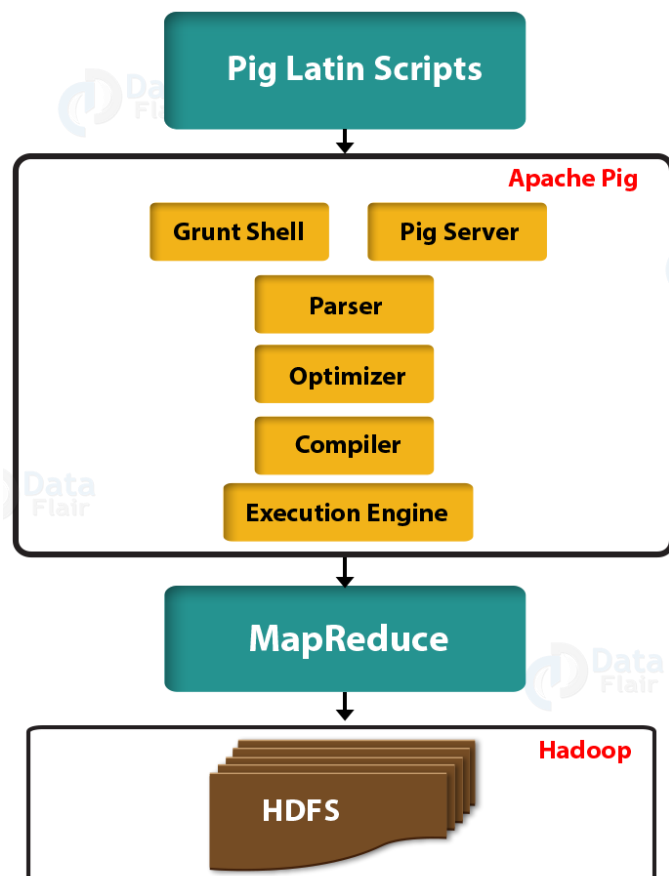
- High-level scripting language called Pig Latin.
- Automatically converts scripts into MapReduce jobs.

- Supports structured, semi-structured, and unstructured data.
- Extensible through User Defined Functions (UDFs).
- Provides built-in operators for filtering, grouping, and sorting.
- Reduces development time compared to Java MapReduce.
- Supports parallel processing on Hadoop clusters.
- Easy integration with Hadoop and HDFS.

Architecture of Pig

Apache Pig consists of several components that convert Pig Latin scripts into executable Hadoop jobs.

1. **Pig Latin Script:** Pig Latin is the high-level scripting language used in Apache Pig. Users write Pig Latin scripts to perform data processing operations such as loading, filtering, grouping, joining, and storing data. These scripts are much simpler than writing Java MapReduce programs and are automatically converted into executable jobs.
2. **Parser:** The Parser is the first component that processes the Pig Latin script. It checks the script for syntax errors, data type compatibility, and logical correctness. If any errors are found, the parser generates error messages; otherwise, it creates a logical plan representing the sequence of operations.
 - Validates Pig Latin syntax.
 - Checks data types and schema.
 - Generates a logical execution plan.
 - Reports errors and warnings.
3. **Logical Optimizer:** The Logical Optimizer improves the efficiency of the logical plan generated by the parser. It applies optimization techniques to reduce unnecessary operations and improve execution performance.
 - Removes redundant operations.
 - Combines multiple operations where possible.
 - Optimizes data filtering and projection.
 - Reduces processing time and resource usage.
4. **Compiler:** The Compiler converts the optimized logical plan into a physical execution plan. This plan contains the actual steps required to execute the script on Hadoop.
 - Converts logical plan into physical plan.
 - Creates MapReduce, Tez, or Spark jobs.
 - Determines execution sequence.
 - Prepares jobs for distributed processing.



5. Execution Engine

The Execution Engine executes the compiled jobs on the Hadoop cluster. It manages job scheduling, task execution, and coordination with Hadoop components.

- Executes MapReduce, Tez, or Spark jobs.
- Monitors job execution.
- Collects and returns results.
- Manages distributed processing.

6. Hadoop Cluster (MapReduce/Tez/Spark): The Hadoop processing framework performs the actual computation on large datasets. Pig can use MapReduce, Apache Tez, or Apache Spark as the execution backend.

- Processes large-scale data in parallel.
- Executes data transformation operations.
- Distributes tasks across cluster nodes.
- Provides scalability and fault tolerance.

7. HDFS (Hadoop Distributed File System): HDFS is the storage layer used by Pig. Input data is read from HDFS, and output results are stored back into HDFS after processing.

- Stores large datasets across multiple nodes.
- Supplies input data to Pig jobs.
- Stores processed output data.
- Provides fault-tolerant storage.

Working of Pig Architecture

1. User writes a Pig Latin script.
2. The Parser validates the script and creates a logical plan.
3. The Logical Optimizer optimizes the plan.
4. The Compiler converts the optimized plan into executable jobs.
5. The Execution Engine submits jobs to Hadoop.
6. Hadoop processes the data stored in HDFS.
7. Results are generated and stored back in HDFS.

Advantages of Pig

- Simplifies Hadoop programming.
- Requires less code than MapReduce.
- Supports rapid application development.
- Handles large-scale data efficiently.
- Provides built-in data transformation functions.
- Easy debugging and testing using Grunt shell.
- Supports extensibility through UDFs.

Grunt

Grunt is the interactive command-line shell of Apache Pig. It provides an environment where users can enter and execute Pig Latin commands interactively without creating a script file. Grunt is mainly used for testing, debugging, and executing Pig operations on Hadoop data.

- Interactive shell for Apache Pig.
- Executes Pig Latin commands directly.
- Useful for learning and testing Pig scripts.
- Allows quick data analysis and debugging.
- Supports execution of Pig scripts from files.
- Interactive execution of Pig Latin commands.
- Supports loading and processing data from HDFS.
- Easy testing and debugging of Pig scripts.
- Executes Pig scripts using the run and exec commands.
- Provides command history and error reporting.
- Supports file system operations.
- Allows viewing schemas and execution plans.

Features of Grunt

- Helps developers quickly validate data transformations.

Advantages of Grunt

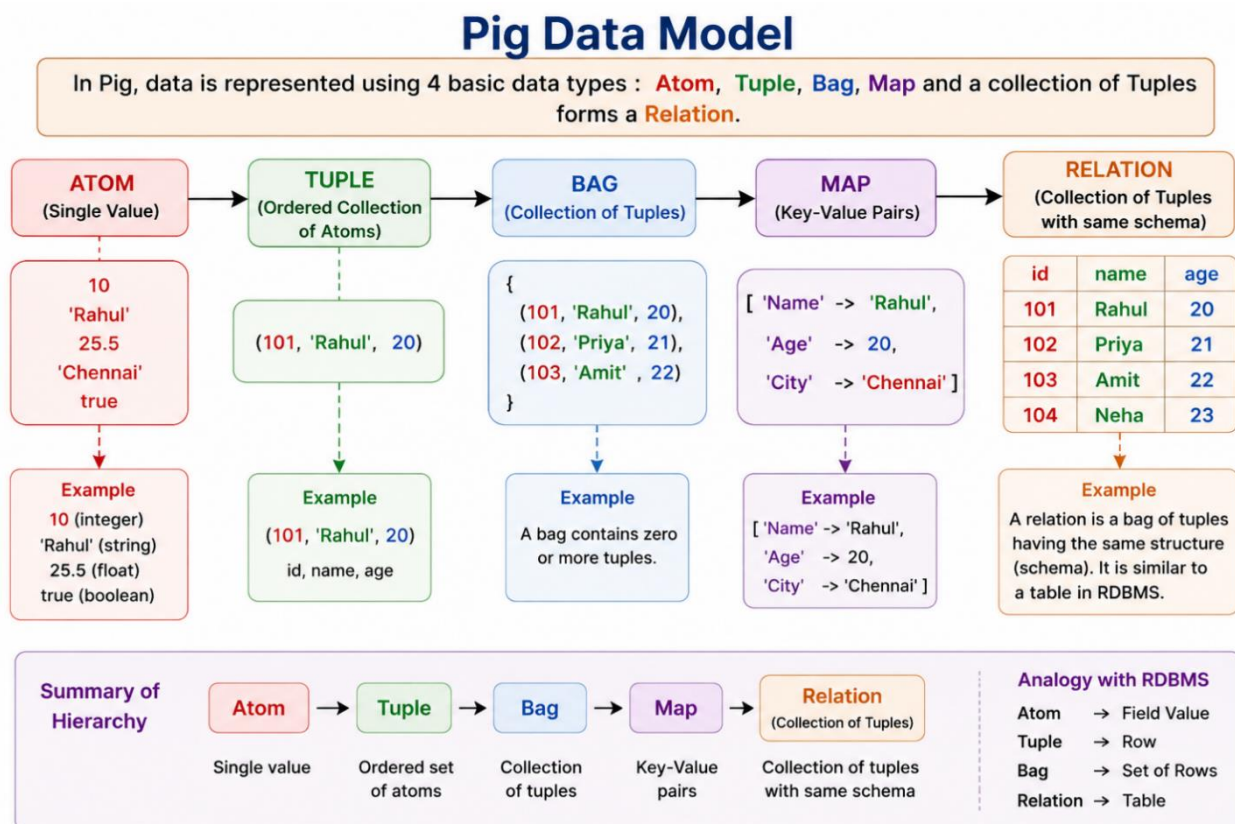
- User-friendly interface.

- Faster development and testing.
- Reduces debugging effort.
- No need to compile programs.
- Suitable for exploratory data analysis.

Pig Data Model

The Pig Data Model defines how data is represented and processed in Apache Pig. It consists of four basic data types: **Atom**, **Tuple**, **Bag**, and **Map**. A collection of tuples forms a **Relation**.

1. **Atom:** An **Atom** is the simplest data element in Pig. It contains a single value and cannot be divided further.
 - Smallest unit of data.
 - Represents a single value.
 - Similar to a field in a database table.
 - Can be integer, float, string, etc.
2. **Tuple:** A **Tuple** is an ordered set of fields enclosed within parentheses ().
 - Contains multiple atoms.
 - Similar to a row in a database table.
 - Fields may have different data types.
 - Ordered collection of values.
3. **Bag:** A **Bag** is a collection of tuples enclosed within curly braces {}.
 - Collection of tuples.
 - Tuples need not be unique.
 - Similar to a table containing rows.
 - Most common data structure in Pig.
4. **Map:** A **Map** stores data as key-value pairs.
 - Contains key-value pairs.
 - Keys are strings.
 - Values can be any data type.
 - Useful for semi-structured data.



5. **Relation:** A **Relation** is a collection of tuples having the same schema. It is the fundamental data structure used in Pig Latin.
 - Collection of tuples.
 - Similar to a database table.

- Stored as a bag internally.
- Supports operations such as filtering, grouping, and joining
- .

Pig Latin

Pig Latin is a high-level data flow language used in Apache Pig for processing large datasets. It provides a simple way to express data analysis tasks without writing complex MapReduce programs. Pig Latin scripts are automatically translated into MapReduce, Tez, or Spark jobs and executed on a Hadoop cluster. Pig Latin follows a **procedural programming approach**, where users specify a sequence of operations to be performed on data.

- High-level scripting language for Hadoop.
- Simplifies big data processing.
- Converts scripts into MapReduce jobs automatically.
- Supports structured, semi-structured, and unstructured data.
- Suitable for ETL (Extract, Transform, Load) operations.

Features of Pig Latin

1. **Simplicity:** Pig Latin is much easier to learn than Java MapReduce.
2. **Reduced Coding:** Performs complex data processing tasks using fewer lines of code.
3. **Extensibility:** Supports custom User Defined Functions written in Java, Python, or JavaScript.
4. **Parallel Processing:** Automatically runs operations in parallel across Hadoop clusters.
5. **Rich Operators:** Includes built-in tools for filtering, grouping, sorting, and joining data.
6. **Schema Flexibility:** Handles diverse data formats with or without defined schemas.
7. **Optimization:** Automatically tweaks execution plans to deliver better performance.
8. **Scalability:** Efficiently processes massive terabyte and petabyte-scale datasets.

Common Operators

Operator	Purpose
LOAD	Load data
FILTER	Select records
FOREACH	Process records
GROUP	Group data
ORDER	Sort data

DISTINCT	Remove duplicates
JOIN	Combine datasets
UNION	Merge datasets
LIMIT	Restrict output
STORE	Save results

User Defined Functions: A **User Defined Function (UDF)** is a custom function created by users to perform operations that are not available in Pig's built-in functions.

Advantages of UDFs

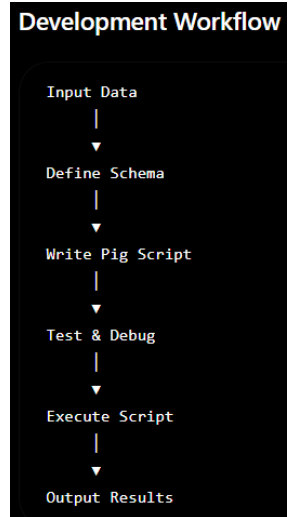
- Extends Pig functionality.
- Supports custom business logic.
- Reusable across multiple Pig scripts.
- Improves flexibility.

Languages Supported

- Java (most common)
- Python
- JavaScript
- Groovy

Developing and Testing Pig Latin Scripts

1. **Identify Data Source:** Determine the input data stored in HDFS or local storage.
2. **Define Schema:** Specify the field names and data types for the data.
3. **Write Pig Latin Script:** Use Pig operators to process the input data.
4. **Validate Logic:** Check the correctness of the code using sample data.
5. **Execute Script:** Run the final verified script using the Pig engine.
6. **Analyze Results:** Verify the output data and optimize the system's performance.



Module 6.2: Hive

Contents

Apache Hive	1
Hive Data Types.....	2
Hive File Formats.....	3
Similarities between Pig and Hive.....	6
Differences between Pig and Hive	6

Apache Hive

Apache Hive is a data warehouse and SQL-like query tool built on top of Hadoop. It allows users to analyze and manage large datasets stored in HDFS using HiveQL, a language similar to SQL. Hive converts queries into MapReduce, Tez, or Spark jobs for execution.

- Provides SQL-like query language (HiveQL).
- Supports large-scale data processing.
- Runs on top of Hadoop and HDFS.
- Supports partitioning and bucketing.
- Allows schema-on-read data management.
- Supports various file formats such as ORC, Parquet, and Avro.
- Extensible through User Defined Functions (UDFs).

Advantages of Hive

- Easy SQL-like querying of big data.
- Eliminates complex MapReduce programming.
- Scalable for petabyte-scale datasets.
- Supports data warehousing operations.
- Integrates with Hadoop ecosystem tools.
- Supports multiple file formats.

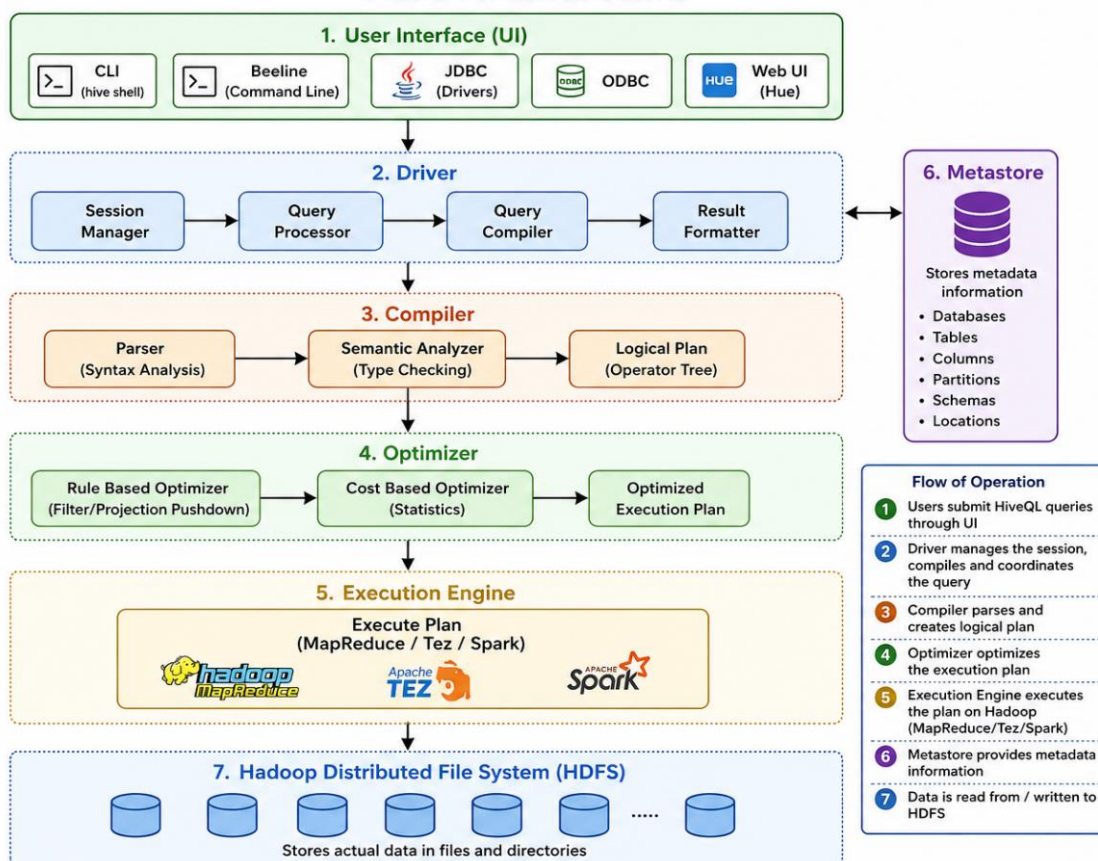
Applications of Hive

- Data warehousing.
- Business intelligence reporting.
- Log file analysis.
- ETL (Extract, Transform, Load) processing.
- Data summarization.
- Big data analytics.

Hive Architecture

1. **User Interface (UI):** Provides access to Hive through tools like CLI, Beeline, JDBC, or Web UI.
2. **Driver:** Receives HiveQL queries, manages user sessions, and controls the overall execution.
3. **Compiler:** Parses queries, checks syntax rules, and generates a logical execution plan.
4. **Optimizer:** Modifies and refines the execution plan to deliver better overall performance.
5. **Execution Engine:** Runs the optimized plan using frameworks like MapReduce, Tez, or Spark.
6. **Metastore:** Stores metadata including table names, column schemas, partitions, and data locations.
7. **Hadoop Ecosystem:** Connects directly with HDFS for data storage and Hadoop for distributed processing.

Hive Architecture



Hive Data Types

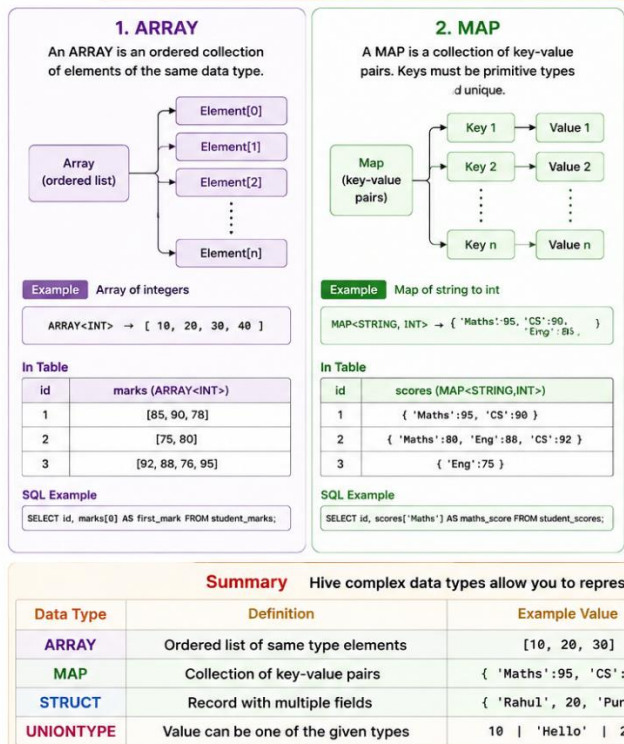
Hive supports both primitive and complex data types.

1. Primitive Data Types

Data Type	Description	Example
TINYINT	1-byte integer	10
SMALLINT	2-byte integer	200
INT	4-byte integer	1000
BIGINT	8-byte integer	100000
FLOAT	Single precision decimal	12.5
DOUBLE	Double precision decimal	25.678
DECIMAL	Exact numeric value	123.45
BOOLEAN	True or False	TRUE
STRING	Character string	"Hive"
CHAR	Fixed length string	CHAR(10)
VARCHAR	Variable length string	VARCHAR(50)
DATE	Date value	2025-01-01
TIMESTAMP	Date and time	2025-01-01 10:00:00

Hive Complex

Hive supports complex data types that allow you to store colle



Hive File Formats

Hive supports several storage formats for efficient data processing.

- Text File:** Default storage format in Hive where data is stored as plain text.
 - Human readable.
 - Easy to create and edit.
 - Larger storage size.
 - Slower query performance.
- Sequence File:** Binary key-value file format used in Hadoop.
 - Stores data in compressed form.
 - Faster than text files.
 - Suitable for MapReduce jobs.
- ORC File (Optimized Row Columnar):** A highly optimized columnar storage format developed for Hive.
 - High compression.
 - Faster query execution.
 - Supports indexing.
 - Reduces storage requirements.

Advantages

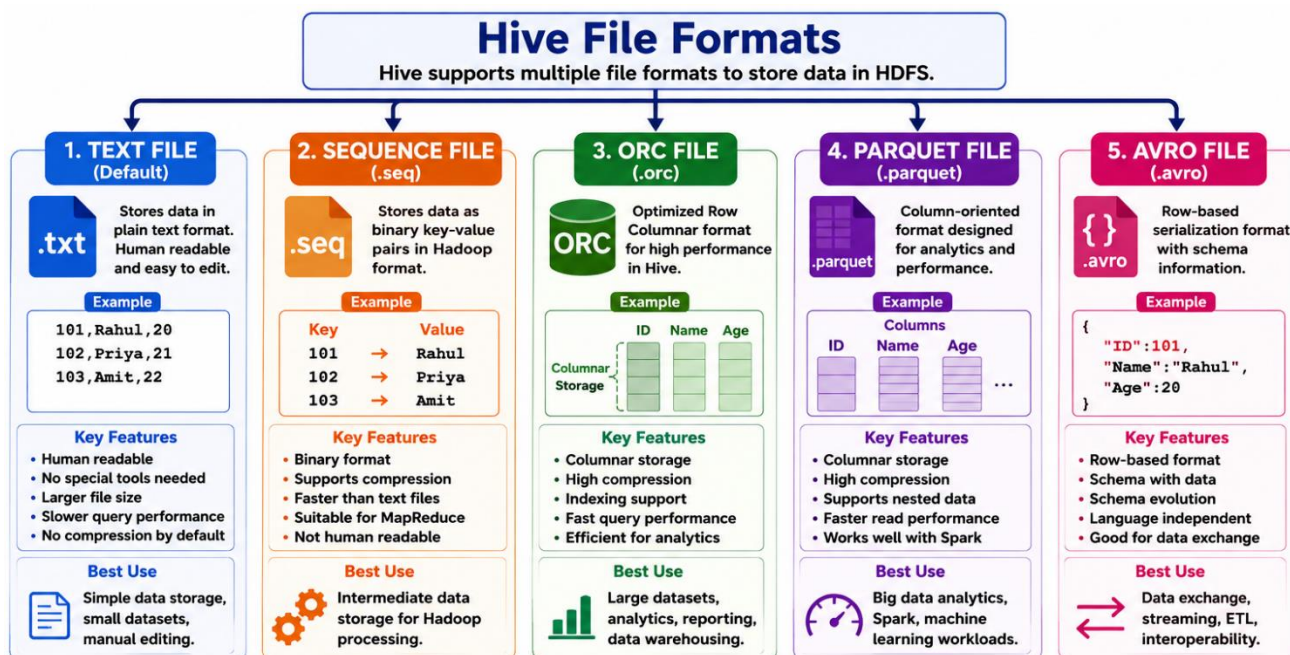
 - Excellent performance for analytical queries.
 - Efficient reading of selected columns.
- Parquet File:** A column-oriented storage format widely used in big data systems.
 - Efficient compression.
 - Faster analytical processing.
 - Supports complex nested data structures.

Advantages

 - Works efficiently with Hive and Spark.
 - Reduces disk I/O.
- Avro File:** A row-based data serialization format developed by Apache.
 - Schema stored with data.
 - Supports schema evolution.
 - Language independent.

Advantages

 - Good for data exchange between systems.
 - Suitable for streaming and ETL applications.



Format	Storage Type	Compression	Query Speed	Best Use
Text File	Row	Low	Slow	Simple data storage
Sequence File	Key-Value	Medium	Moderate	Hadoop processing
ORC	Columnar	High	Very Fast	Hive analytics
Parquet	Columnar	High	Very Fast	Hive & Spark analytics
Avro	Row	Medium	Moderate	Data exchange & ETL

HiveQL DDL, DML and Queries

6.3.4 HIVEQL DATA DEFINITION LANGUAGE (DDL)

6.3.4.1 Creating Databases

Syntax

```
CREATE DATABASE database_name;
```

Example

```
CREATE DATABASE CollegeDB;
```

Verify

```
SHOW DATABASES;
```

6.3.4.2 Creating Tables

Syntax

```
CREATE TABLE table_name (  
  column1 datatype,  
  column2 datatype,  
  column3 datatype  
);
```

Example

```
CREATE TABLE Student (  
  ID INT,  
  Name STRING,  
  Age INT  
);
```

View Structure: DESCRIBE Student;

6.3.4.3 Altering Tables

Add Column

```
ALTER TABLE Student  
ADD COLUMNS (Department STRING);
```

Rename Table

```
ALTER TABLE Student  
RENAME TO Student_Info;
```

Change Column Name

```
ALTER TABLE Student_Info  
CHANGE Age Student_Age INT;
```

6.3.4.4 Dropping Databases and Tables

Drop Table

```
DROP TABLE Student_Info;
```

Drop Database

```
DROP DATABASE CollegeDB;
```

Force Drop Database

```
DROP DATABASE CollegeDB CASCADE;
```

Purpose: Used to permanently remove tables and databases from Hive.

6.3.5.1 Loading Data

Syntax

```
LOAD DATA INPATH 'filepath'  
INTO TABLE table_name;
```

Example

```
LOAD DATA INPATH '/user/hadoop/student.txt'  
INTO TABLE Student;
```

Purpose: Imports external data into Hive tables.

6.3.5.2 Inserting Data

Insert Single Record

```
INSERT INTO Student  
VALUES (101, 'Rahul', 20);
```

Insert Multiple Records

```
INSERT INTO Student VALUES  
(102, 'Priya', 21),  
(103, 'Amit', 22);
```

Purpose: Adds new records into a table.

6.3.5.3 Updating Data

(Update supported in ACID transactional tables)

Example

```
UPDATE Student  
SET Age = 21  
WHERE ID = 101;
```

Purpose: Modifies existing records.

6.3.5.4 Deleting Data

(Delete supported in ACID transactional tables)

Example

```
DELETE FROM Student  
WHERE ID = 101;
```

Purpose: Removes unwanted records from a table.

6.3.6 HIVEQL QUERIES

6.3.6.1 SELECT Query

Example

```
SELECT * FROM Student;
```

Output

ID	Name	Age
101	Rahul	20
102	Priya	21
103	Amit	22

Purpose: Retrieves data from a table.

6.3.6.2 WHERE Clause

Example

```
SELECT * FROM Student  
WHERE Age > 20;
```

Output

ID	Name	Age
102	Priya	21
103	Amit	22

Purpose: Filters records based on condition.

6.3.6.3 GROUP BY Clause

Example

```
SELECT Department, COUNT(*)  
FROM Employee  
GROUP BY Department;
```

Output

Department	Count
CSE	50
ECE	40

Purpose: Groups rows having the same values.

6.3.6.4 ORDER BY Clause

Example

```
SELECT * FROM Student  
ORDER BY Age DESC;
```

Output

ID	Name	Age
103	Amit	22
102	Priya	21
101	Rahul	20

Purpose: Sorts query results.

6.3.6.5 JOIN Operations

Example (INNER JOIN)

```
SELECT S.ID, S.Name, M.Marks  
FROM Student S  
JOIN Marks M  
ON S.ID = M.ID;
```

Output

ID	Name	Marks
101	Rahul	85
102	Priya	90

Purpose: Combines rows from two or more tables.

6.3.6.6 Aggregate Functions

Example

```
SELECT  
COUNT(*) AS Total_Students,  
AVG(Age) AS Avg_Age,  
MAX(Age) AS Max_Age,  
MIN(Age) AS Min_Age  
FROM Student;
```

Output

Total_Students	Avg_Age	Max_Age	Min_Age
3	21.00	22	20

Purpose: Performs calculations on multiple rows.

Summary

DDL: Defines structure (DB, Tables)

DML: Manages data (Load, Insert, Update, Delete)

Queries: Retrieves and analyzes data

HiveQL is SQL-like language used to interact with Hive data stored in HDFS.



Similarities between Pig and Hive

Apache Pig and Apache Hive are high-level data processing tools developed for the Hadoop ecosystem. Both simplify big data processing by providing abstraction over MapReduce programming.

1. **Built on Hadoop:** Both Pig and Hive run on Hadoop and use HDFS for data storage. They convert user commands into MapReduce, Tez, or Spark jobs.
2. **Handle Large Data Sets:** Designed to process and analyze massive volumes of structured, semi-structured, and unstructured data.
3. **Reduce Coding Complexity:** Eliminate the need to write complex Java MapReduce programs.
4. **Support Distributed Processing:** Data is processed across multiple nodes in a Hadoop cluster.
5. **Extensible:** Both support User Defined Functions (UDFs) for custom operations.
6. **Fault Tolerant:** Inherit Hadoop's fault tolerance and reliability features.
7. **Open Source:** Both are Apache open-source projects and freely available.
8. **Data Transformation Capabilities:** Provide operations for filtering, grouping, sorting, and joining data.

Differences between Pig and Hive

Feature	Pig	Hive
Developed By	Yahoo	Facebook
Language Used	Pig Latin	HiveQL (SQL-like language)
User Type	Programmers and Developers	Data Analysts and SQL Users
Programming Style	Procedural	Declarative
Learning Curve	Requires scripting knowledge	Easy for SQL users
Data Processing	Describes how data flows	Describes what result is needed
Query Language	Pig Latin	HiveQL
Schema	Schema can be applied when needed	Schema is defined before querying
Best For	ETL and data transformation	Data warehousing and reporting
Optimization	Limited automatic optimization	Built-in query optimization
Database Support	No database concepts	Supports databases and tables
Output	Processed datasets	Query results and reports
Ease of Use	More flexible for programmers	More user-friendly for analysts

Use Cases of Pig

1. **ETL (Extract, Transform, Load) Operations:** Cleaning and transforming raw data before loading into databases.
2. **Log File Processing:** Analyzing web server logs and application logs.
3. **Data Integration:** Combining data from multiple sources.
4. **Data Preparation:** Preparing datasets for machine learning and analytics.
5. **Large-Scale Data Transformation:** Filtering, grouping, sorting, and aggregating large datasets.

Example Applications

- Social media data analysis
- Clickstream analysis
- Data cleansing pipelines
- Recommendation system preprocessing

Use Cases of Hive

1. **Data Warehousing:** Storing and managing structured big data.
2. **Business Intelligence (BI):** Generating reports and dashboards.
3. **Data Analysis:** Querying large datasets using SQL-like commands.
4. **Reporting Systems:** Creating periodic business reports.
5. **Decision Support Systems:** Analyzing historical data for strategic decisions.

Example Applications

- Sales reporting
- Customer analytics
- Financial analysis
- Data warehouse management
- Business dashboard

